

SKRIPSI

**ANALISIS PREDIKTIF JUMLAH KASUS DEMAM
BERDARAH DENGUE DENGAN MODEL PEMBELAJARAN
MESIN XGBOOST DAN LSTM**



DYLAN RICHARD

NPM: 6162001168

**PROGRAM STUDI MATEMATIKA
FAKULTAS SAINS
UNIVERSITAS KATOLIK PARAHYANGAN
2025**

FINAL PROJECT

PREDICTIVE ANALYSIS OF DENGUE FEVER CASES USING XGBOOST AND LSTM MACHINE LEARNING MODELS



DYLAN RICHARD

NPM: 6162001168

DEPARTMENT OF MATHEMATICS
FACULTY OF SCIENCE
PARAHYANGAN CATHOLIC UNIVERSITY
2025

LEMBAR PENGESAHAN

ANALISIS PREDIKTIF JUMLAH KASUS DEMAM BERDARAH DENGUE DENGAN MODEL PEMBELAJARAN MESIN XGBOOST DAN LSTM

DYLAN RICHARD

NPM: 6162001168

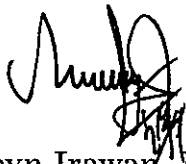
Telah lulus ujian skripsi pada 17 Juli 2025 dengan penguji:
Agus Sukmana, M.Sc. dan Steven Sergio, M.Aktr.

Bandung, 8 Agustus 2025

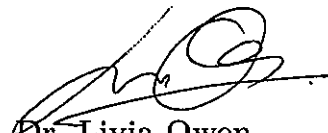
Menyetujui,

Pembimbing 1

Pembimbing 2



Robyn Irawan, M.Sc.



Dr. Livia Owen

Mengetahui,

Ketua Program Studi



Jonathan Hoseana, Ph.D.

PERNYATAAN

Dengan ini saya yang bertandatangan di bawah ini menyatakan bahwa skripsi dengan judul:

ANALISIS PREDIKTIF JUMLAH KASUS DEMAM BERDARAH DENGUE DENGAN MODEL PEMBELAJARAN MESIN XGBOOST DAN LSTM

adalah benar-benar karya saya sendiri, dan saya tidak melakukan penjiplakan atau pengutipan dengan cara-cara yang tidak sesuai dengan etika keilmuan yang berlaku dalam masyarakat keilmuan.

Atas pernyataan ini, saya siap menanggung segala risiko dan sanksi yang dijatuhkan kepada saya, apabila di kemudian hari ditemukan adanya pelanggaran terhadap etika keilmuan dalam karya saya, atau jika ada tuntutan formal atau non-formal dari pihak lain berkaitan dengan keaslian karya saya ini.

Dinyatakan di Bandung,
Tanggal 8 Agustus 2025



Dylan Richard
NPM: 6162001168

ABSTRAK

Demam Berdarah Dengue adalah penyakit menular yang disebabkan oleh virus dengue dan ditularkan oleh nyamuk *Aedes aegypti*. Di Indonesia, penyakit ini menjadi masalah serius bagi kesehatan masyarakat, dengan jumlah kasus yang terus meningkat setiap tahun. Dalam skripsi ini, dibandingkan dua model pembelajaran mesin, yaitu *Long Short-Term Memory* (LSTM) dan *Extreme Gradient Boosting* (XGBoost), dalam memprediksi kasus DBD di Indonesia. Model dibangun menggunakan data historis jumlah kasus DBD serta variabel-variabel sosiodemografis. Data diolah melalui proses pembersihan untuk menghilangkan nilai yang hilang atau tidak relevan, kemudian dilakukan transformasi agar format data sesuai dengan kebutuhan model, dan selanjutnya dibagi menjadi data latih serta data uji untuk mengevaluasi performa model secara lebih objektif. Hasil penelitian menunjukkan bahwa LSTM memiliki performa yang lebih baik dan lebih stabil dibandingkan XGBoost dalam memprediksi jumlah kasus DBD. Selain itu, variabel yang terkait dengan fasilitas kesehatan ditemukan sebagai variabel yang memiliki kontribusi tertinggi dalam model prediksi.

Kata-kata kunci: DBD; prediksi; pembelajaran mesin; LSTM; XGBoost.

ABSTRACT

Dengue fever is an infectious disease caused by the dengue virus and transmitted by the *Aedes aegypti* mosquito. In Indonesia, this disease poses a serious public health problem, with the number of cases increasing every year. This thesis compares two machine learning models, namely Long Short-Term Memory (LSTM) and Extreme Gradient Boosting (XGBoost), in predicting dengue cases in Indonesia. The models were developed using historical data on dengue case numbers and sociodemographic variables. The data were processed through a cleaning stage to remove missing or irrelevant values, followed by data transformation to meet the model requirements, and then divided into training and testing datasets to objectively evaluate the model's performance. The results indicate that LSTM demonstrates better and more stable performance compared to XGBoost in predicting the number of dengue cases. Additionally, variables related to healthcare facilities were found to have the highest contribution in the prediction model.

Keywords: Dengue fever; prediction; machine learning; LSTM; XGBoost.

It does not matter how slowly you go as long as you do not stop.
-Confucius



KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan berkat dan penyertaan-Nya yang tiada henti, sehingga penulis dapat menyelesaikan skripsi yang berjudul **Analisis Prediktif Jumlah Kasus Demam Berdarah Dengue dengan Model Pembelajaran Mesin XGBoost dan LSTM**. Skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana pada Program Studi Matematika, Fakultas Sains, Universitas Katolik Parahyangan. Penulis menyampaikan terima kasih sebesar-besarnya kepada semua pihak yang telah memberikan bimbingan, dukungan, dan semangat selama proses penyusunan skripsi ini. Secara khusus, penulis mengucapkan terima kasih kepada:

1. Ibu Dr. Livia Owen dan Bapak Robyn Irawan, M.Sc., selaku dosen pembimbing yang telah dengan sabar memberikan arahan, masukan, serta motivasi yang sangat berarti selama proses penelitian dan penulisan skripsi ini;
2. Bapak Dr. Andreas Parama Wijaya selaku dosen wali yang senantiasa memberikan bimbingan dan dukungan selama masa studi;
3. Bapak Agus Sukmana, M.Sc. dan Bapak Steven Sergio, M.Aktr., selaku dosen penguji yang telah memberikan masukan yang membangun demi penyempurnaan skripsi ini;
4. Keluarga saya, terutama kedua adik saya, atas doa, semangat, dan dukungan yang terus mengalir sejak awal hingga akhir penyusunan skripsi ini;
5. Teman-teman komsel di Elshadai Community Church yang telah menjadi sumber kekuatan rohani dan dukungan moral selama proses perkuliahan dan penyusunan skripsi;
6. Teman-teman angkatan 2020, 2021, dan 2022 yang tidak dapat disebutkan satu per satu, namun telah menjadi bagian penting dalam perjalanan akademik penulis melalui semangat kebersamaan, bantuan, dan dukungan yang sangat berarti;
7. Santa Meliana, yang telah menjadi rekan bimbingan selama setengah tahun pertama di bawah arahan dosen pembimbing, serta telah memberikan dukungan dan semangat melalui kebersamaan dalam proses awal penyusunan skripsi.

Penulis menyadari bahwa skripsi ini masih memiliki kekurangan. Oleh karena itu, segala bentuk saran dan masukan yang membangun akan sangat dihargai. Semoga skripsi ini dapat memberikan manfaat bagi pembaca dan pengembangan ilmu pengetahuan di bidang analisis data dan pembelajaran mesin.

Bandung, 8 Agustus 2025

Dylan Richard

DAFTAR ISI

KATA PENGANTAR	viii
DAFTAR ISI	ix
DAFTAR GAMBAR	xi
DAFTAR TABEL	xii
DAFTAR NOTASI	1
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan	2
1.4 Sistematika Pembahasan	3
2 LANDASAN TEORI	4
2.1 Pembelajaran Mesin	4
2.2 Pohon Regresi	4
2.3 <i>Random Forest</i>	5
2.4 <i>Gradient Boosting</i>	6
2.5 <i>Extreme Gradient Boosting</i> (XGBoost)	8
2.6 <i>Artificial Neural Network</i> (ANN)	10
2.7 <i>Recurrent Neural Network</i> (RNN)	15
2.8 <i>Long Short-Term Memory</i> (LSTM)	16
2.9 Metrik Evaluasi Model	18
3 IMPLEMENTASI MODEL XGBOOST DAN LSTM	19
3.1 Pengumpulan dan Prapengolahan Data	19
3.2 Proses Prapengolahan Data	21
3.3 Paramater-Parameter XGBoost	23
3.4 Paramater-Parameter LSTM	23
3.5 Perhitungan Manual XGBoost	23
3.6 Perhitungan Manual LSTM	34
3.7 Diagram Alir Pemodelan Data XGBoost dan LSTM	42
4 HASIL DAN PEMBAHASAN	44
4.1 Hasil Analisis Menggunakan XGBoost	44
4.2 Hasil Analisis Menggunakan LSTM	47
4.3 Perbandingan Evaluasi XGBoost dan LSTM	48
5 KESIMPULAN DAN SARAN	50
5.1 Kesimpulan	50
5.2 Saran	51



DAFTAR GAMBAR

2.1	Struktur umum pohon keputusan.	5
2.2	Diagram skema XGBoost	8
2.3	Ilustrasi arsitektur ANN yang terdiri dari tiga lapisan.	11
2.4	Ilustrasi <i>forward propagation</i>	12
2.5	Struktur jaringan RNN.	15
2.6	Ilustrasi sel LSTM.	16
3.1	Diagram alir pemodelan data menggunakan XGBoost dan LSTM.	43
4.1	Perbandingan MSE <i>loss</i> antara data pelatihan dan data validasi.	48



DAFTAR TABEL

3.1	Contoh data	21
3.2	Contoh <i>One-Hot Encoding</i> pada Area	22
3.3	Data IPM, Kepadatan Penduduk, Persentase Puskesmas, dan Jumlah Kasus DBD	24
3.4	Residual prediksi awal Jumlah Kasus DBD	24
3.5	<i>Gain</i> ketiga variabel	25
3.6	Prediksi baru setelah pemisahan pertama	26
3.7	Perhitungan residual berdasarkan prediksi pertama (F_1)	26
3.8	Prediksi baru setelah pemisahan kedua	27
3.9	Perhitungan residual berdasarkan prediksi kedua (F_2)	27
3.10	Prediksi baru setelah pemisahan ketiga	28
3.11	Perhitungan residual berdasarkan prediksi ketiga (F_3)	28
3.12	Hasil prediksi dan residual pada setiap pemisahan di pohon pertama	28
3.13	Prediksi dan residual awal pohon kedua	29
3.14	Prediksi baru setelah pemisahan pertama	29
3.15	Perhitungan residual berdasarkan prediksi pertama (F_1)	30
3.16	Prediksi baru setelah pemisahan kedua	30
3.17	Perhitungan residual berdasarkan prediksi kedua (F_2)	30
3.18	Prediksi baru setelah pemisahan ketiga	31
3.19	Perhitungan residual berdasarkan prediksi ketiga (F_3)	31
3.20	Hasil prediksi dan residual pada setiap pemisahan di pohon kedua	32
3.21	Residual akhir pada kedua pohon XGBoost	32
3.22	Nilai <i>Min</i> dan <i>Max</i> masing-masing variabel	34
3.23	Nilai standardisasi IPM	34
3.24	Nilai standardisasi Kepadatan Penduduk	35
3.25	Nilai standardisasi Persentase Puskesmas	35
3.26	Data standardisasi IPM, Kepadatan Penduduk, dan Persentase Puskesmas	35
4.1	Kombinasi parameter yang digunakan dalam pelatihan XGBoost	44
4.2	Hasil terbaik XGBoost pada berbagai variasi <i>cross-validation</i>	44
4.3	Nilai kontribusi variabel berdasarkan XGBoost	45
4.4	Kombinasi parameter yang digunakan dalam pelatihan LSTM	47
4.5	Hasil terbaik LSTM berdasarkan nilai MSE	47
4.6	Perbandingan evaluasi nilai MSE pada XGBoost dan LSTM	49

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Demam Berdarah Dengue (DBD) adalah penyakit menular yang disebabkan oleh virus dengue dan ditularkan melalui nyamuk *Aedes aegypti*. Penyakit ini menjadi salah satu masalah kesehatan masyarakat yang serius di berbagai negara tropis dan subtropis, termasuk Indonesia. DBD dapat menyebabkan gejala berupa demam tinggi, nyeri sendi, sakit kepala, ruam kulit, hingga kondisi yang lebih parah seperti pendarahan dan kegagalan organ yang berpotensi fatal jika tidak ditangani dengan baik. ¹

Setiap tahun, jumlah kasus DBD terus meningkat di seluruh dunia, termasuk di Indonesia, yang dipengaruhi oleh berbagai variabel sosiodemografis, seperti tingkat urbanisasi, akses ke fasilitas kesehatan, dan kepadatan penduduk yang berkontribusi terhadap penyebaran penyakit. ² Kondisi ini semakin memburuk dengan pertumbuhan populasi yang pesat, yang menciptakan lingkungan ideal bagi nyamuk *Aedes aegypti* berkembang biak, terutama di daerah yang memiliki tingkat urbanisasi dan kepadatan penduduk yang tinggi. Selain itu, keterbatasan akses ke fasilitas kesehatan menjadi penentu karena kurangnya fasilitas kesehatan dapat menghambat deteksi awal dan penanganan yang cepat. Oleh karena itu, upaya pengendalian DBD menjadi fokus utama dalam menjaga kesehatan masyarakat.

Salah satu solusi untuk mengatasi permasalahan ini adalah dengan memanfaatkan model pembelajaran mesin dalam analisis prediktif. Hingga saat ini, model analisis prediktif banyak digunakan untuk memprediksi penyakit menular lain [1]. Analisis prediktif memanfaatkan data historis yang berkaitan dengan berbagai variabel penyebaran DBD untuk menghasilkan prediksi yang lebih akurat. Dengan menggabungkan data-data tersebut, model prediksi akan mampu mengidentifikasi pola dan tren yang tidak terlihat secara kasat mata dan memberikan wawasan yang lebih komprehensif mengenai risiko penularan di masa depan.

Penelitian oleh Wardhana, Wang, dan Sibuea [1] membahas penerapan lima model pembelajaran mesin untuk memprediksi tingkat kasus penyakit di Indonesia, yaitu *Logistic Regression*, *Decision Tree*, *Random Forest*, *Support Vector Machine* (SVM), dan *Extreme Gradient Boosting* (XGBoost). Masing-masing model diterapkan untuk memprediksi lima jenis penyakit berbeda, termasuk penyakit DBD. Penelitian ini menggunakan data sosiodemografis pada tahun 2016 hingga 2021. Namun, penelitian tersebut hanya menyajikan hasil akurasi tanpa melakukan analisis lebih lanjut terhadap

¹dr. Meva Nareza T, "Demam Berdarah", Alodokter, <https://www.alodokter.com/demam-berdarah>.

²Nurul Hayat, "Perdoki Sebut Tingkat Kasus DBD Masih Tinggi di Indonesia", Antara, <https://www.antaranews.com/berita/4482589/perdoki-sebut-tingkat-kasus-dbd-masih-tinggi-di-indonesia>.

kontribusi tiap variabel terhadap hasil prediksi. Sementara itu, penelitian oleh Nurillah, Imrona, dan Alamsyah [2] menggunakan LSTM untuk memprediksi penyakit DBD berdasarkan data historis dari Rumah Sakit Umum Daerah Besemah sebanyak 9.061 data kasus, dengan rentang tahun 2015 hingga 2019. Selain data jumlah kasus, penelitian tersebut juga menggunakan data suhu, kelembapan udara, dan curah hujan yang diperoleh dari BMKG. Adapun dalam skripsi ini, digunakan data yang serupa dengan penelitian oleh Wardhana, Wang, dan Sibuea, yaitu data sosiodemografis, namun pendekatannya diperluas dengan menambahkan model LSTM serta melakukan analisis kontribusi variabel terhadap hasil prediksi menggunakan XGBoost.

Dalam skripsi ini, akan membandingkan dua model, yaitu *Long Short-Term Memory* (LSTM) dan *Extreme Gradient Boosting* (XGBoost). Melalui perbandingan ini, akan diketahui kelebihan dan kelemahan model dalam memprediksi kasus DBD di Indonesia. Selain itu, skripsi ini akan mengetahui variabel-variabel yang berpengaruh dalam memprediksi jumlah kasus DBD. Hasil dari skripsi yang telah dilakukan diharapkan dapat memberikan kontribusi dalam pengembangan sistem prediksi yang lebih akurat, serta menjadi acuan untuk mendukung strategi pencegahan dan penanggulangan DBD yang lebih efektif di Indonesia.

1.2 Rumusan Masalah

Rumusan masalah yang dibahas pada skripsi ini dapat dirumuskan sebagai berikut:

1. Bagaimana cara memodelkan dan memprediksi jumlah kasus DBD dengan model pembelajaran mesin dengan memanfaatkan data historis dan variabel-variabel sosiodemografis?
2. Variabel-variabel apa saja yang paling kontribusi terhadap prediksi jumlah kasus DBD di suatu wilayah?
3. Bagaimana perbandingan performa XGBoost dan LSTM yang diterapkan dalam memprediksi jumlah kasus DBD?

1.3 Tujuan

Berdasarkan rumusan masalah yang telah dipaparkan, tujuan penulisan skripsi ini adalah sebagai berikut:

1. Memodelkan dan memprediksi jumlah kasus DBD dengan memanfaatkan data historis dan variabel sosiodemografis untuk menghasilkan prediksi yang lebih akurat.
2. Menggunakan nilai *feature importance* untuk mengetahui variabel yang berkontribusi terhadap prediksi jumlah kasus DBD di suatu wilayah.
3. Menggunakan nilai *Mean Squared Error* (MSE) untuk membandingkan performa XGBoost dan LSTM dalam memprediksi jumlah kasus DBD.

1.4 Sistematika Pembahasan

Pengerjaan skripsi ini dibagi ke dalam beberapa bab, yaitu bab pendahuluan, bab landasan teori, bab pendekatan dan prosedur penelitian, bab hasil dan pembahasan, dan bab kesimpulan dan saran. Sistematika penulisan skripsi ini adalah sebagai berikut:

Bab I : Pendahuluan

Bab ini membahas mengenai latar belakang dari penelitian dalam skripsi ini. Selain itu, bagian ini juga membahas rumusan masalah, tujuan, dan batasan masalah.

Bab II : Landasan Teori

Bab ini membahas teori-teori dasar yang mendukung penyusunan makalah ini. Teori yang akan dibahas meliputi Pohon Regresi, *Random Forest*, *Gradient Boosting*, *Extreme Gradient Boosting* (XGBoost), *Artificial Neural Network* (ANN), *Recurrent Neural Network* (RNN), dan *Long Short-Term Memory* (LSTM).

Bab III : Implementasi Model XGBoost dan LSTM

Bab ini membahas proses dari pengumpulan data hingga pemodelan data. Pembahasan pada bab ini akan menjelaskan implementasi XGBoost dan LSTM dalam pembangunan model.

Bab IV : Hasil dan Pembahasan

Bab ini membahas hasil dan pembahasan dari proses pemodelan yang telah dilakukan menggunakan XGBoost dan LSTM. Pembahasan mencakup kombinasi parameter yang digunakan, perbandingan setiap kombinasi parameter, hasil pemodelan dengan parameter terbaik, evaluasi kinerja model, serta analisis lebih lanjut terhadap kontribusi variabel pada XGBoost dan grafik MSE *loss* selama pelatihan pada LSTM.

Bab V : Kesimpulan dan Saran

Bab ini membahas kesimpulan dari hasil penelitian yang telah dilakukan, berdasarkan analisis XGBoost dan LSTM dalam memprediksi jumlah kasus DBD. Selain itu, bab ini juga memuat saran yang dapat dijadikan acuan untuk penelitian selanjutnya.

BAB 2

LANDASAN TEORI

Pada bab ini akan dibahas teori-teori pendukung yang digunakan dalam skripsi ini. Teori yang dibahas meliputi Pohon Regresi, *Random Forest*, *Gradient Boosting*, *Extreme Gradient Boosting* (XGBoost), *Artificial Neural Network* (ANN), *Recurrent Neural Network* (RNN), dan *Long Short-Term Memory* (LSTM).

2.1 Pembelajaran Mesin

Pembelajaran mesin adalah bidang dari kecerdasan buatan (*artificial intelligence*) yang digunakan untuk belajar dan membuat keputusan. Menurut Ian Goodfellow [3], pembelajaran mesin adalah studi tentang algoritma di mana performa suatu algoritma meningkat seiring dengan pengalaman yang diperoleh. Algoritma pembelajaran mesin dimulai dengan memproses data di mana algoritma tersebut mempelajari pola untuk membuat suatu prediksi. tersebut akan dioptimalkan seiring dengan banyaknya data yang diproses. Misalkan dalam prediksi cuaca, algoritma mempelajari data historis cuaca untuk memberikan ramalan yang akurat di masa depan. Berdasarkan data yang digunakan, pembelajaran mesin dibagi menjadi dua, yaitu *supervised learning* dan *unsupervised learning*.

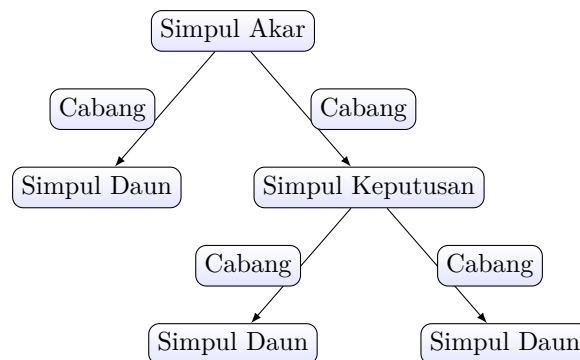
Supervised learning merupakan metode pembelajaran mesin di mana model dilatih untuk memahami hubungan antara variabel bebas dan variabel terikat. Pendekatan ini terutama digunakan untuk melakukan prediksi dengan memanfaatkan pola hubungan yang telah teridentifikasi dalam data. Sementara itu, *unsupervised learning* adalah metode pembelajaran mesin yang berfokus pada pencarian pola dalam data tanpa adanya informasi mengenai variabel terikat. Tujuan utama dari *unsupervised learning* adalah mengidentifikasi struktur data secara mandiri tanpa bergantung pada hasil prediksi.

Pada *supervised learning*, metode yang digunakan dibagi menjadi dua tergantung pada jenis data variabel terikat, yaitu klasifikasi dan regresi. Jika data variabel terikat bersifat numerik, maka metode yang digunakan adalah regresi. Sedangkan jika data variabel terikat bersifat kategorikal, maka metode yang digunakan adalah klasifikasi. Untuk skripsi ini, data yang digunakan bersifat numerik sehingga metode yang digunakan adalah regresi.

2.2 Pohon Regresi

Pohon keputusan adalah algoritma dalam pembelajaran mesin yang berbentuk pohon untuk membuat model prediksi berdasarkan serangkaian keputusan. Pohon keputusan dapat digunakan untuk kasus

klasifikasi maupun regresi. Karena data yang digunakan bertipe numerik, maka lebih cocok menggunakan pohon regresi untuk memprediksi nilai kontinu berdasarkan pemisahan data ke dalam bagian-bagian yang lebih kecil.



Gambar 2.1: Struktur umum pohon keputusan.

Seperti ditunjukkan pada Gambar 2.1, pohon keputusan dimulai dari simpul akar, tempat data pertama diproses, lalu dipisah di setiap simpul keputusan berdasarkan kondisi tertentu. Data kemudian mencapai simpul daun, di mana nilai target aktual y digunakan untuk menghitung prediksi akhir ($\hat{y}_i$) sebagai rata-rata semua nilai yang terikat dalam simpul tersebut. Setiap data yang masuk ke simpul daun dibedakan dengan indeks i , di mana i merujuk pada setiap data yang berada di simpul daun. Cabang-cabang dalam pohon merepresentasikan proses pemisahan data berdasarkan aturan yang diterapkan di setiap simpul hingga akhirnya mencapai simpul daun.

Pohon regresi adalah salah satu metode dari pohon keputusan yang dibentuk melalui suatu algoritma penyekatan (*if-then logical*) secara rekursif. Untuk menentukan kriteria percabangan, algoritma akan memilih penyekatan yang paling optimal untuk meminimumkan ketidakakuratan prediksi dengan membagi data menjadi dua kelompok berdasarkan nilai atribut tertentu. Proses ini terus berulang sampai mencapai kesalahan prediksi yang minimal atau jumlah data di dalam simpul terlalu sedikit untuk dibagi lebih lanjut.

Pohon regresi cenderung menjadi terlalu dalam dan kompleks yang dapat menyebabkan *overfitting*. Oleh karena itu, *pruning* digunakan sebagai teknik untuk mengurangi kompleksitas model dengan memotong di cabang yang tidak memberikan kontribusi signifikan dalam prediksi. Hal ini dapat menghasilkan model yang lebih akurat dalam menghasilkan sebuah prediksi [4, hlm. 331-332].

Setelah proses *pruning*, model dievaluasi agar tidak hanya baik pada data latih tetapi juga pada data baru. Salah satu metode yang umum digunakan adalah *K-fold cross-validation*, yang memungkinkan setiap bagian data berperan sebagai data latih maupun data uji, sehingga penggunaan data menjadi maksimal [4, hlm. 333]. Hasil setiap iterasi kemudian dievaluasi menggunakan *Mean Square Error* (MSE) untuk menilai kualitas prediksi, yang akan dibahas lebih lanjut pada Subbab 2.9.

2.3 *Random Forest*

Random Forest adalah algoritma *supervised learning* yang dikeluarkan oleh Breiman pada tahun 2001 [5]. Algoritma ini digunakan untuk menyelesaikan masalah yang berhubungan dengan klasifikasi, regresi, dan sebagainya. Ada dua hal yang membuat algoritma ini disebut *random*, yaitu:

1. Setiap pohon dibangun dengan sampel data *bootstrap* yang diambil menggunakan cara *bootstrap*, yaitu pengambilan sampel dari data latih secara acak dengan pengembalian.
2. Untuk setiap pemilahan simpul selama pembentukan pohon keputusan, salah satu dari m variabel bebas dipilih secara acak dan digunakan sebagai dasar pemisahan di simpul tersebut.

Algoritma ini berupa kombinasi dari beberapa pohon keputusan di mana setiap pohon bergantung pada nilai vektor acak yang dijadikan sampel secara bebas dan merata pada semua pohon dalam *forest* tersebut. Hasil prediksi akhir ($\hat{\mathbf{y}}$) dari *Random Forest* didapatkan melalui rata-rata prediksi dari setiap pohon keputusan yang dibangun dalam model, sedangkan $\mathbf{y}$ merujuk pada vektor fitur (*input*) yang digunakan oleh model untuk melakukan prediksi. Untuk *Random Forest* yang memiliki jumlah N pohon dapat dirumuskan sebagai berikut:

$$\hat{\mathbf{y}} = \frac{1}{N} \sum_{n=1}^N h_n(\mathbf{y}).$$

di mana $h_n(\mathbf{y})$ adalah prediksi dari pohon ke- n untuk *input* $\mathbf{y}$. *Random Forest* memiliki mekanisme internal untuk mengestimasi *generalization error*, yaitu melalui error *out-of-bag* (OOB). Saat membentuk pohon, sekitar $\frac{2}{3}$ data digunakan dalam sampel *bootstrap*, sementara $\frac{1}{3}$ sisanya digunakan untuk menguji performa pohon tersebut. Estimasi OOB dihitung sebagai rata-rata kesalahan prediksi, di mana tiap kasus $\mathbf{y}$ dievaluasi hanya oleh pohon-pohon yang tidak menyertakan $\mathbf{y}$ dalam sampel *bootstrap* [6].

2.4 Gradient Boosting

Gradient Boosting adalah algoritma dalam pembelajaran mesin yang digunakan untuk tugas klasifikasi maupun regresi, di mana model pohon keputusan dibangun secara bertahap. Setiap model baru dibuat untuk memperbaiki kesalahan dari model sebelumnya, hingga diperoleh model akhir yang memenuhi kriteria tertentu [7]. Algoritma ini dikenal karena mampu menghasilkan performa tinggi, fleksibel dalam menangani berbagai jenis data, serta memungkinkan pengaturan parameter untuk menghindari *overfitting*.

Gradient Boosting bekerja dengan menyesuaikan model dasar secara bertahap, di mana setiap model baru dibentuk untuk mengurangi kesalahan prediksi sebelumnya melalui penurunan nilai *loss function*. *Loss function* mengukur perbedaan antara hasil prediksi dan nilai sebenarnya. Semakin kecil nilai *loss function*, maka semakin baik pula performa model dalam melakukan prediksi. Gradien menunjukkan arah terbaik untuk meminimumkan nilai tersebut, sehingga akurasi meningkat seiring pelatihan. Pendekatan ini mengoptimalkan pohon keputusan pada setiap simpul terminal. Untuk menjalankan *Gradient Boosting*, diperlukan beberapa *input* utama, yaitu:

1. Data pelatihan yang terdiri atas pasangan variabel bebas dan variabel terikat (x_i, y_i) untuk $i = 1, 2, \dots, n$, di mana x_i adalah variabel bebas untuk data ke- i dan y_i adalah variabel bebas untuk data ke- i .
2. *Loss function* $L(y_i, F(x_i))$ yang diminimumkan, di mana $F(x_i)$ adalah hasil gabungan dari model-model sebelumnya terhadap data x_i . Nilai *loss function* digunakan untuk menghitung gradien, yang selanjutnya menjadi dasar dalam proses pembaruan model pada setiap iterasi.

3. Jumlah iterasi M yang menentukan banyaknya langkah pembaruan model.
4. *Learning rate* β yang digunakan untuk mengontrol kontribusi setiap iterasi terhadap model akhir.

Langkah-langkah *Gradient Boosting* dapat dijelaskan sebagai berikut:

1. Nilai konstanta awal model $F_0(\mathbf{x})$ ditentukan dengan rumus:

$$F_0(\mathbf{x}) = \arg \min \sum_{i=1}^N L(y_i, F(x_i)),$$

di mana $L(y_i, F(x_i))$ adalah *loss function* yang dihitung berdasarkan kesalahan antara nilai target y_i dan prediksi model $F(x_i)$ pada data ke- i . Dalam hal ini, x_i merepresentasikan vektor fitur input dari observasi ke- i .

2. Untuk setiap iterasi dari $m = 1$ hingga M , gradien residual dihitung sebagai berikut:

$$g_i = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(\mathbf{x})=F_{m-1}(\mathbf{x})}, \quad \forall i \in 1, 2, \dots, N,$$

di mana g_i adalah gradien residual dan $F_{m-1}(\mathbf{x})$ adalah model pada iterasi sebelumnya. Dalam hal ini, $\mathbf{x}$ merepresentasikan vektor fitur *input* dari sebuah data.

3. Pengklasifikasi dasar digunakan untuk menyesuaikan data residu. Dengan menggunakan pendekatan kuadrat terkecil, parameter dari model dihitung dan model $h_m(\mathbf{x})$ dioptimalkan untuk meminimumkan kesalahan antara prediksi dan data residu. Model $h_m(\mathbf{x})$ diperoleh dengan cara sebagai berikut:

$$h_m(\mathbf{x}) = \arg \min_h \sum_{i=1}^N [g_i - h(x_i)]^2,$$

4. *Loss function* diminimumkan untuk menentukan ukuran langkah baru β_m :

$$\beta_m = \arg \min_{\beta} \sum_{i=1}^N L(y_i, F_{m-1}(x_i) + \beta \cdot h_m(x_i)),$$

5. Model kemudian diperbarui dengan:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \beta_m \cdot h_m(\mathbf{x}).$$

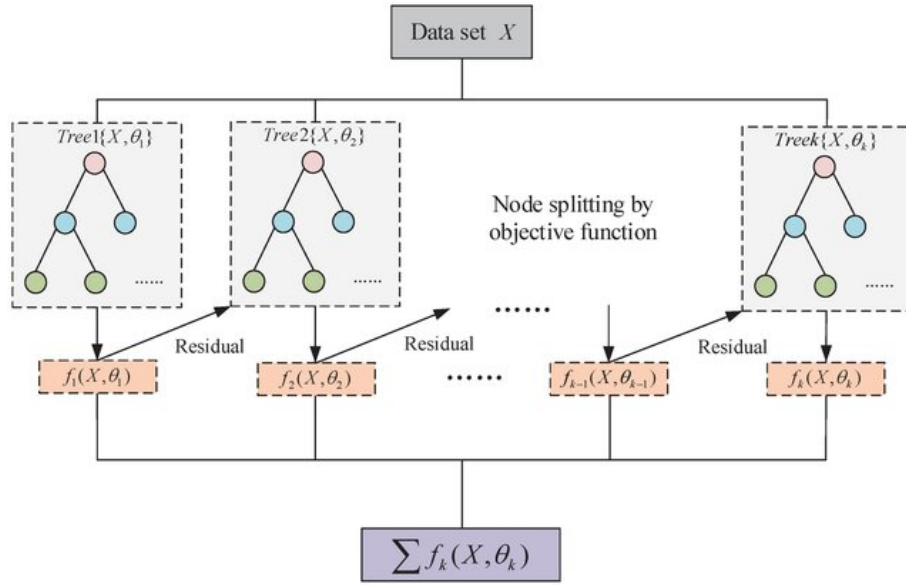
Interpretasi dan contoh perhitungan akan diberikan pada Subbab 3.5.

Algoritma 1 *Gradient Boosting*

- 1: **Masukan:** banyaknya data (N), variabel bebas (x_i), variabel terikat (y_i), jumlah iterasi (M), *loss function* ($L(y_i, F(x_i))$), *learning rate* (β), parameter regularisasi (λ).
 - 2: $F_0(\mathbf{x}) = \arg \min \sum_{i=1}^N L(y_i, F(x_i))$.
 - 3: Untuk $m = 1, \dots, M$ lakukan:
 - 4: $g_i = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(\mathbf{x})=F_{m-1}(\mathbf{x})}, \quad i \in \{1, 2, \dots, N\}$.
 - 5: $h_m(\mathbf{x}) = \arg \min_h \sum_{i=1}^N [g_i - h(x_i)]^2$.
 - 6: $\beta_m = \arg \min_{\beta} \sum_{i=1}^N L(y_i, F_{m-1}(x_i) + \beta \cdot h_m(x_i))$.
 - 7: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \beta_m \cdot h_m(\mathbf{x})$.
 - 8: **Keluaran:** Fungsi $F_m(\mathbf{x})$ setelah M iterasi.
-

2.5 Extreme Gradient Boosting (XGBoost)

Extreme Gradient Boosting (XGBoost) adalah algoritma pembelajaran mesin yang dikembangkan oleh Tianqi Chen pada tahun 2016 [8]. XGBoost merupakan pengembangan dari *Gradient Boosting* yang dirancang untuk meningkatkan efisiensi dan mencegah *overfitting* melalui teknik seperti regularisasi dan *pruning*. Selain itu, XGBoost mampu membagi proses pengolahan data menjadi beberapa bagian yang dijalankan secara bersamaan melalui paralelisasi, serta menyebarkan data ke beberapa sistem secara terpisah dengan distribusi komputasi. Hal ini membuat XGBoost lebih efisien saat menangani data dalam jumlah besar.



Gambar 2.2: Diagram skema XGBoost [9].

Pada Gambar 2.2, model dibangun secara bertahap seperti pada model *Gradient Boosting*, di mana setiap langkah atau model baru dirancang untuk memperbaiki kesalahan prediksi yang dibuat oleh model sebelumnya. Proses ini berlangsung hingga residunya diminimumkan. Namun, berbeda dengan *Gradient Boosting*, XGBoost meminimumkan fungsi objektif ($L(\phi)$) dengan menambahkan regularisasi ($\Omega(f_k)$) untuk mengatur kompleksitas model, dimana ϕ merujuk pada kumpulan dari semua pohon keputusan yang telah dibangun dan dioptimalkan dalam model. Fungsi objektif dirumuskan sebagai berikut:

$$L(\phi) = \sum_{i=1}^n L(y_i, F(x_i)) + \sum_{k=1}^K \Omega(f_k),$$

di mana $\Omega(f_k)$ merupakan fungsi regularisasi yang bertujuan untuk mengontrol kompleksitas model agar tidak terjadi *overfitting*. Fungsi ini didefinisikan sebagai berikut:

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \|w\|^2,$$

di mana:

1. γT : penalti yang mengatur jumlah total simpul daun (T) dalam pohon keputusan. Parameter γ menentukan seberapa besar penalti yang diberikan untuk setiap simpul daun yang ada.

2. $\frac{1}{2}\lambda\|w\|^2$: parameter regularisasi yang mengatur besarnya penalti pada bobot simpul daun (w) dalam pohon keputusan.

Dalam setiap iterasi, XGBoost memperbarui prediksi berdasarkan hasil pohon sebelumnya untuk mengatur kontribusi model baru. Proses ini dioptimalkan secara bertahap dengan memanfaatkan informasi gradien. Untuk meningkatkan stabilitas dan efisiensi, XGBoost menerapkan ekspansi Taylor hingga gradien kedua, sehingga fungsi objektif menghasilkan persamaan (2.1).

$$\begin{aligned} L(\phi)^{(t)} &= \sum_{i=1}^n L(y_i, F_{m-1}(x_i)) + \sum_{k=1}^K \Omega(f_k) \\ &\approx \sum_{i=1}^n \left[L(y_i, F_{m-1}(x_i)) + g_i h_m(x_i) + \frac{1}{2} h_i h_m^2(x_i) \right] + \sum_{k=1}^K \Omega(f_k), \end{aligned} \quad (2.1)$$

Pada persamaan (2.1), gradien pertama (g_i) dari *loss function* menunjukkan arah perubahan yang optimal dalam memperbarui prediksi, sedangkan gradien kedua (h_i) mengontrol besarnya langkah pembaruan agar lebih stabil. Dengan mempertimbangkan turunan kedua, XGBoost dapat meningkatkan stabilitas optimasi dan mempercepat proses mencari solusi optimal karena memiliki informasi tambahan mengenai perubahan gradien. Selain itu, penggunaan gradien kedua (h_i) membantu dalam regularisasi dengan membatasi perubahan bobot pada setiap simpul daun, sehingga model lebih tahan terhadap *overfitting*.

Dalam skripsi ini, karena permasalahan yang dibahas berkaitan dengan regresi, maka *loss function* yang digunakan adalah MSE. Pada fungsi MSE, turunan pertama terhadap prediksi menghasilkan nilai gradien $g_i = \hat{y}_i - y_i$, sedangkan turunan keduanya menghasilkan nilai konstan $h_i = 1$. Karena $h_i = 1$, maka *loss function* memberikan respon yang sama untuk setiap data, tanpa mempertimbangkan perbedaan antar data. Dengan kata lain, nilai (h_i) yang konstan menyederhanakan perhitungan optimasi karena tidak ada variasi dalam pengaruh perubahan prediksi terhadap fungsi loss di antara data yang berbeda.

Pada algoritma XGBoost, pemilihan pemisahan pada setiap pohon dilakukan dengan cara memaksimalkan nilai *Gain* dari fungsi objektif. Fungsi *Gain* dihitung untuk menentukan pemisahan terbaik yang mampu mengurangi kesalahan prediksi secara optimal. *Gain* untuk suatu pemisahan dihitung dengan rumus sebagai berikut:

$$Gain = \frac{\left(\sum_{i \in I_L} g_i \right)^2}{\left(\sum_{i \in I_L} h_i \right) + \lambda} + \frac{\left(\sum_{i \in I_R} g_i \right)^2}{\left(\sum_{i \in I_R} h_i \right) + \lambda} - \frac{\left(\sum_{i \in I} g_i \right)^2}{\left(\sum_{i \in I} h_i \right) + \lambda},$$

di mana:

1. I_L dan I_R : data untuk cabang kiri dan cabang kanan setelah pemisahan.
2. I : data untuk simpul induk.
3. g_i : jumlah gradien pada cabang kiri dan kanan setelah pemisahan.
4. h_i : jumlah data pada cabang kiri dan kanan setelah pemisahan.

Setelah pemisahan dilakukan, bobot setiap simpul daun dihitung untuk meminimumkan *loss function* dengan cara mengurangi kesalahan pada cabang-cabang yang terbentuk. Bobot untuk

simpul daun pada pohon ke- j (w_j) dihitung menggunakan rumus berikut:

$$w_j = -\frac{\sum_{i \in I_j} g_i}{\left(\sum_{i \in I_j} h_i\right) + \lambda}, \quad j = L, R,$$

di mana I_j adalah data yang berada pada simpul daun ke- j , dengan L dan R merujuk pada cabang kiri dan kanan yang dihasilkan setelah pemisahan. Dengan menghitung bobot tersebut, pohon keputusan dapat menyesuaikan nilai pada setiap cabang berdasarkan kontribusinya dalam mengurangi kesalahan model. Proses ini membantu mengoptimalkan fungsi objektif secara keseluruhan dan menghasilkan model yang tidak hanya akurat, tetapi juga mampu menangani kompleksitas data dengan baik [10].

Setelah bobot simpul daun dihitung, prediksi model akan diperbarui untuk mengurangi kesalahan prediksi dari iterasi sebelumnya. Proses ini akan menambahkan kontribusi dari pohon baru ke model sebelumnya dengan menggunakan rumus berikut:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \beta \cdot w_j.$$

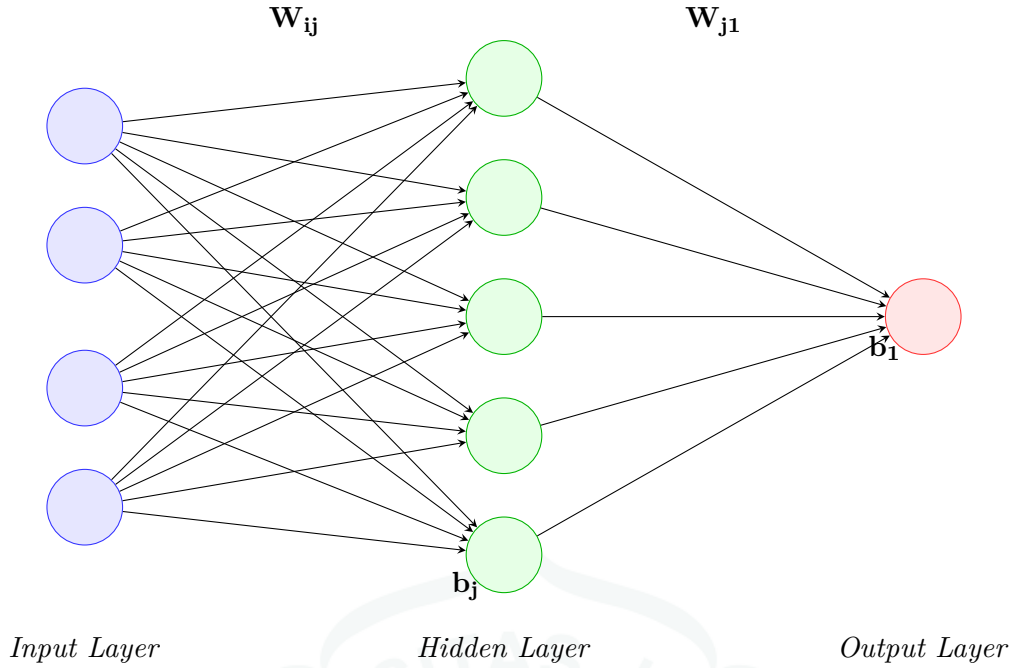
di mana $F_m(\mathbf{x})$ adalah prediksi model pada iterasi ke- m . Setiap pohon baru di XGBoost dirancang untuk meminimumkan kesalahan prediksi dengan fokus pada data yang memiliki nilai residual terbesar melalui perbaikan gradien pertama (g_i), sehingga prediksi model semakin akurat.

Algoritma 2 XGBoost

- 1: **Masukan:** banyaknya data (N), variabel bebas (x_i), variabel terikat (y_i), jumlah iterasi (M), *loss function* ($L(y_i, F(x_i))$), parameter regularisasi (λ), *learning rate* (β).
 - 2: $F_0(\mathbf{x}) = \arg \min \sum_{i=1}^N L(y_i, F(x_i))$.
 - 3: Untuk $m = 1, \dots, M$ lakukan:
 - 4: Untuk $i = 1, \dots, N$ lakukan:
 - 5: $g_i = \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F(\mathbf{x})=F_{m-1}(\mathbf{x})}$.
 - 6: $h_i = \left. \frac{\partial^2 L(y_i, F(x_i))}{\partial F^2(x_i)} \right|_{F(\mathbf{x})=F_{m-1}(\mathbf{x})}$.
 - 7: **Pengulangan selesai.**
 - 8: $Gain = \frac{\left(\sum_{i \in I_L} g_i\right)^2}{\left(\sum_{i \in I_L} h_i\right) + \lambda} + \frac{\left(\sum_{i \in I_R} g_i\right)^2}{\left(\sum_{i \in I_R} h_i\right) + \lambda} - \frac{\left(\sum_{i \in I} g_i\right)^2}{\left(\sum_{i \in I} h_i\right) + \lambda}$
 - 9: $w_j = -\frac{\sum_{i \in I_j} g_i}{\left(\sum_{i \in I_j} h_i\right) + \lambda}, \quad j = L, R.$
 - 10: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \beta \cdot w_j.$
 - 11: **Pengulangan selesai.**
 - 12: **Keluaran:** Fungsi $F_m(\mathbf{x})$ setelah M iterasi.
-

2.6 Artificial Neural Network (ANN)

Artificial Neural Network (ANN) adalah metode pembelajaran mesin yang terinspirasi dari jaringan saraf otak manusia. ANN terdiri dari neuron buatan yang saling terhubung melalui bobot, serta memiliki bias sebagai parameter tambahan. Neuron menerima masukan yang dikalikan dengan bobot, lalu diproses menggunakan fungsi aktivasi untuk menghasilkan keluaran. Fungsi aktivasi ini berperan penting dalam mempelajari hubungan non-linear pada data. Keunggulan ini membuat ANN sering digunakan untuk tugas-tugas seperti klasifikasi citra, prediksi deret waktu, dan pengenalan suara. Ilustrasi arsitektur ANN ditunjukkan pada Gambar 2.3.



Gambar 2.3: Ilustrasi arsitektur ANN yang terdiri dari tiga lapisan.

Pada ANN, terdapat tiga lapisan yang membentuk sebuah struktur dasar ANN, yaitu:

1. Lapisan masukan (*Input Layer*): Lapisan yang menyimpan dan meneruskan data awal ke lapisan berikutnya.
2. Lapisan tersembunyi (*Hidden Layer*): Lapisan yang bertanggung jawab untuk memproses dan mengekstrak informasi dari data yang terdiri dari beberapa lapisan yang saling terhubung.
3. Lapisan keluaran (*Output Layer*): Lapisan yang bertugas menghasilkan keluaran berupa prediksi berdasarkan proses data sebelumnya.

Forward Propagation

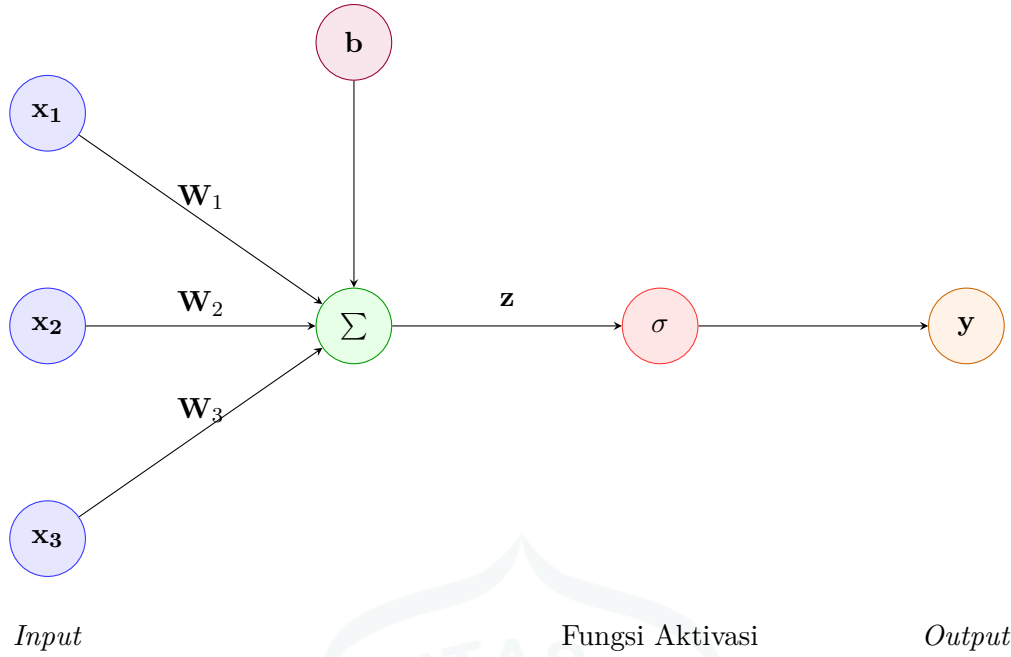
Forward propagation adalah proses dalam *neural network* untuk menghitung *output* dari suatu *input* dengan melewati setiap lapisan. Proses ini melibatkan perhitungan nilai-nilai aktivasi dari setiap neuron berdasarkan bobot, bias, dan fungsi aktivasi yang digunakan. Hasil dari proses ini menjadi dasar untuk mengevaluasi performa model dengan membandingkan nilai *output* yang dihasilkan terhadap nilai target.

Dapat dilihat pada Gambar 2.4, proses dimulai di lapisan masukan yang terdiri dari tiga neuron, yaitu x_1 , x_2 , dan x_3 . Setiap neuron memiliki bobot tersendiri (W_i) dimana dilanjutkan ke lapisan keluaran z dengan bias b yang dihitung dengan rumus sebagai berikut:

$$z = (W_1 \cdot x_1) + (W_2 \cdot x_2) + (W_3 \cdot x_3) + b,$$

Setelah menghitung nilai z , nilai z diteruskan ke fungsi aktivasi untuk mendapatkan nilai *output* neuron (y). Secara umum, fungsi aktivasi dapat dituliskan sebagai berikut:

$$y = \sigma(z),$$

Gambar 2.4: Ilustrasi *forward propagation*.

Secara matematis, perhitungan *forward propagation* ditulis dalam bentuk matriks. Hal ini dilakukan untuk mempermudah komputasi, terutama dalam jaringan yang besar. Melalui operasi matriks, proses perhitungan dapat dilakukan secara efisien dan mengurangi beban komputasi yang terjadi pada *neural network*. Misalnya, pada lapisan ke- k , hasil perhitungan dapat ditulis sebagai berikut:

$$\mathbf{Z}^{(k)} = \mathbf{W}^{(k)} \cdot \mathbf{X}^{(k-1)} + \mathbf{B}^{(k)},$$

di mana $\mathbf{W}^{(k)}$ adalah matriks bobot untuk lapisan ke- k , $\mathbf{X}^{(k-1)}$ adalah vektor aktivasi dari lapisan sebelumnya, dan $\mathbf{B}^{(k)}$ adalah vektor bias.

Setelah menghitung nilai $\mathbf{Z}^{(k)}$ pada setiap lapisan, fungsi aktivasi dapat diterapkan untuk menghasilkan nilai *output* neuron dari lapisan tersebut. Secara matematis, nilai *output* neuron $\mathbf{Y}^{(k)}$ dari lapisan ke- k dihitung dengan menerapkan fungsi aktivasi pada $\mathbf{Z}^{(k)}$ dapat ditulis sebagai berikut:

$$\mathbf{Y}^{(k)} = \sigma(\mathbf{Z}^{(k)}).$$

Dengan demikian, $\mathbf{Y}^{(k)}$ merepresentasikan aktivasi akhir dari lapisan ke- (k) , yang kemudian akan menjadi masukan bagi lapisan berikutnya dalam jaringan saraf.

Fungsi Aktivasi

Fungsi aktivasi dalam ANN adalah komponen penting yang bertugas mengubah *output* linear dari sebuah neuron menjadi *output* non-linear. Fungsi ini menentukan bagaimana sinyal dari setiap neuron diproses sebelum diteruskan ke neuron berikutnya. Pada skripsi ini, fungsi aktivasi yang akan digunakan adalah fungsi aktivasi sigmoid, fungsi aktivasi tanh, dan fungsi aktivasi linear. Berikut adalah penjelasan masing-masing fungsi aktivasi yang akan digunakan:

1. Fungsi Aktivasi Sigmoid

Fungsi aktivasi sigmoid adalah fungsi aktivasi yang mengubah *input* menjadi nilai dalam rentang $(0, 1)$. Fungsi ini sering digunakan sebagai bagian dari perhitungan LSTM agar mencegah peningkatan drastis.

$$\sigma(\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{x}}}.$$

2. Fungsi Aktivasi Tanh

Fungsi aktivasi tanh adalah fungsi aktivasi yang mengubah *input* menjadi nilai dalam rentang $(-1, 1)$. Fungsi ini sering digunakan di *hidden state* untuk mencegah pertumbuhan nilai yang berlebihan yang dapat menyebabkan ketidakstabilan.

$$\tanh(\mathbf{x}) = \frac{e^{\mathbf{x}} - e^{-\mathbf{x}}}{e^{\mathbf{x}} + e^{-\mathbf{x}}}.$$

3. Fungsi Aktivasi Linear

Fungsi aktivasi linear adalah fungsi aktivasi yang memetakan *input* langsung ke *output* tanpa memperkenalkan perubahan skala atau distribusi nilai. Fungsi ini sering digunakan pada *output* untuk tugas regresi.

$$f(\mathbf{x}) = \mathbf{x}.$$

Pada ketiga fungsi di atas, $\mathbf{x}$ menyatakan *input* yang diterima neuron. Nilai ini bisa berupa skalar maupun vektor, tergantung pada banyaknya koneksi dari neuron sebelumnya. Nilai tersebut kemudian diproses oleh fungsi aktivasi untuk menghasilkan keluaran yang akan diteruskan ke neuron berikutnya.

Loss Function

Loss Function adalah fungsi evaluasi yang digunakan untuk mengukur keakuratan keluaran yang dihasilkan oleh model dengan variabel terikat. Karena skripsi ini menggunakan regresi, maka *loss function* yang digunakan adalah *Mean Squared Error* (MSE). Fungsi ini mengukur kesalahan absolut kuadrat antara nilai sebenarnya (y_i) dan nilai prediksi ($\hat{y}_i$). Hal ini membuat MSE menjadi *loss function* yang sering digunakan dalam permasalahan regresi. Namun, kekurangan dari MSE adalah sensitif terhadap nilai pencilan yang mengakibatkan nilai kuadrat error yang semakin besar. *Loss function* untuk MSE dapat dituliskan sebagai berikut:

$$L(\hat{y}_i, y_i) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Back Propagation

Back propagation adalah proses dalam *neural network* yang digunakan untuk memperbarui bobot dan bias berdasarkan kesalahan yang dihitung di lapisan *output*. Proses ini bertujuan untuk meminimumkan *loss function* melalui fungsi gradien dari *loss function*. *Back propagation* melibatkan dua langkah utama, yaitu perhitungan gradien dan pembaharuan bobot yang dilakukan oleh algoritma *optimizer*. Untuk skripsi ini, *loss function* yang akan digunakan adalah MSE dan fungsi aktivasi di lapisan *output* adalah linear. Langkah-langkah perhitungan gradien dengan MSE dapat dijelaskan sebagai berikut:

1. Gradien terhadap Lapisan *Output*.

Gradien dari *loss function* terhadap nilai *output* $\hat{y}_i$ dihitung sebagai berikut. Untuk MSE, gradien terhadap *output* adalah:

$$\frac{\partial L(\hat{y}_i, y_i)}{\partial \hat{y}_i} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i),$$

Karena fungsi aktivasi di lapisan *output* adalah linear, maka nilai keluaran $\hat{y}_i$ sama dengan nilai sebelum aktivasi $\mathbf{z}_i$. Oleh karena itu, turunan dari $\hat{y}_i$ terhadap $\mathbf{z}_i$ adalah:

$$\frac{\partial \hat{y}_i}{\partial \mathbf{z}_i} = 1,$$

Jadi, gradien terhadap $\mathbf{z}_i$ di lapisan *output* adalah:

$$\frac{\partial L(\hat{y}_i, y_i)}{\partial \mathbf{z}_i} = \frac{\partial L(\hat{y}_i, y_i)}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial \mathbf{z}_i} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i),$$

2. Gradien terhadap Bobot dan Bias di Lapisan *Output*.

Setelah menghitung gradien terhadap $\mathbf{z}_i$, gradien terhadap bobot dan bias di lapisan *output* dapat dihitung sebagai berikut:

Untuk bobot $\mathbf{W}$:

$$\frac{\partial L(\hat{y}_i, y_i)}{\partial \mathbf{W}} = \frac{\partial L(\hat{y}_i, y_i)}{\partial \mathbf{z}_i} \cdot \mathbf{x},$$

Untuk bias $\mathbf{b}$:

$$\frac{\partial L(\hat{y}_i, y_i)}{\partial \mathbf{b}} = \frac{\partial L(\hat{y}_i, y_i)}{\partial \mathbf{z}_i}.$$

Optimizer

Optimizer adalah metode yang digunakan untuk memperbarui parameter bobot dan bias agar meminimumkan *loss function* selama proses pelatihan. Salah satu metode yang umum digunakan untuk memperbarui parameter bobot dan bias adalah *gradient descent*. *Gradient descent* bekerja secara bertahap dengan menghitung turunan atau gradien dari *loss function* terhadap parameter, lalu memperbarui parameter tersebut ke arah negatif gradien untuk mendekati nilai minimum *loss function*. Persamaan *gradient descent* dapat ditulis sebagai berikut:

$$\theta_{t+1} = \theta_t - \beta \cdot \left[\frac{\partial L}{\partial \theta_j} \right], \quad j \in \{0, 1, \dots, d\},$$

di mana β adalah *learning rate*, θ adalah parameter, dan d menyatakan banyaknya parameter yang diperbarui selama proses pelatihan.

Pemilihan *optimizer* dan *learning rate* yang optimal sangat penting dalam *gradient descent* karena memengaruhi kecepatan dan hasil pelatihan. Skripsi ini menggunakan algoritma *Adaptive Moment Estimation (Adam)* karena dapat menyesuaikan *learning rate* secara adaptif berdasarkan rata-rata dan kuadrat gradien. Kemampuan ini meningkatkan stabilitas dan efisiensi pelatihan, dengan langkah-langkah algoritma Adam dirangkum sebagai berikut.

1. Perbarui momen pertama ($\mathbf{m}_t$) dan momen kedua ($\mathbf{v}_t$).

$$\mathbf{m}_t = \beta_1 \cdot \mathbf{m}_{t-1} + (1 - \beta_1) \cdot \left[\frac{\partial L}{\partial \theta_j} \right],$$

$$\mathbf{v}_t = \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \cdot \left[\frac{\partial L}{\partial \theta_j} \right]^2,$$

2. Perbaiki momen bias pertama ($\hat{\mathbf{m}}_t$) dan bias kedua ($\hat{\mathbf{v}}_t$).

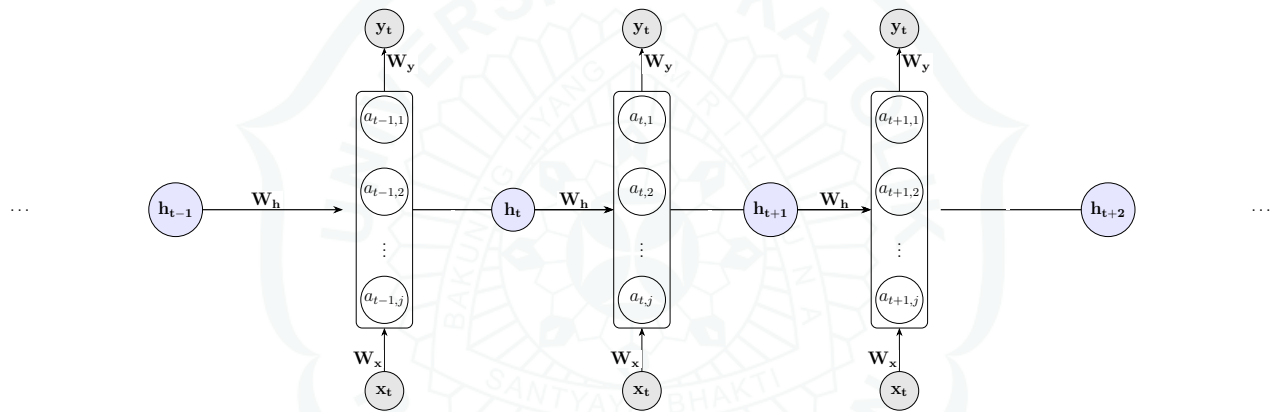
$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t},$$

3. Parameter kemudian diperbarui dengan:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon}}.$$

2.7 Recurrent Neural Network (RNN)

Recurrent Neural Network (RNN) adalah jenis *neural network* yang dirancang untuk memproses data secara berurutan. Tidak seperti ANN, RNN dapat menyimpan informasi dari langkah sebelumnya. Kemampuan ini membuat RNN efektif dalam mengenali pola temporal atau ketergantungan antar data dalam urutan [11, hlm. 271–273].



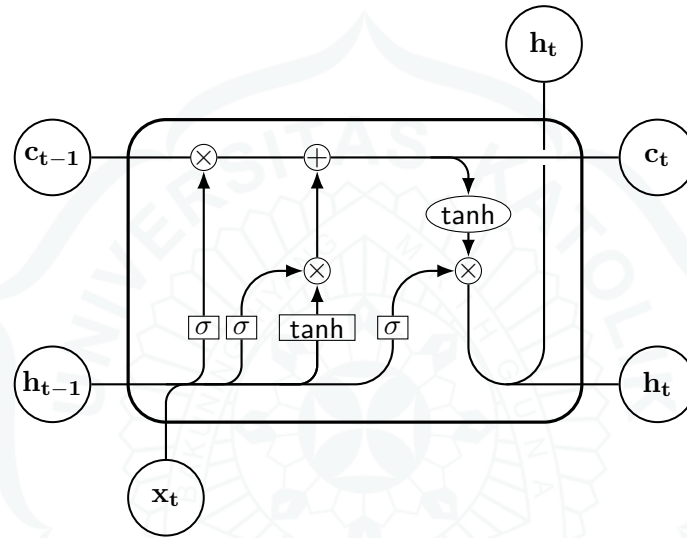
Gambar 2.5: Struktur jaringan RNN.

Dapat dilihat pada Gambar 2.5, proses pengolahan RNN dimulai dengan menerima masukan pada setiap langkah waktu ke- t , yang diproses bersamaan dengan *hidden state* dari langkah sebelumnya. *Hidden state* ($\mathbf{h}_t$) berfungsi sebagai memori internal dimana informasi dari data sebelumnya akan dibawa ke urutan selanjutnya. Setiap neuron dalam *hidden state* memiliki fungsi aktivasi $a_{t,j}$ yang merepresentasikan elemen ke- j dari vektor aktivasi pada waktu ke- t . Selain itu, RNN menggunakan bobot ($\mathbf{W}$) yang membuat data tersebut efisien dalam menangkap pola berulang dalam urutan data.

Akan tetapi, RNN memiliki kelemahan utama dimana gradien cenderung mengecil seiring bertambahnya waktu (*vanishing gradient*) atau gradien menjadi terlalu besar (*exploding gradient*) selama *backward propagation*. Hal ini menghambat proses pembelajaran pada setiap langkah, terutama pada data dengan ketergantungan jangka panjang. Akibatnya, RNN kurang efektif dalam menangkap hubungan antar elemen yang berjauhan dalam urutan data. Untuk mengatasi masalah ini, berbagai varian dari RNN telah dikembangkan, salah satunya adalah *Long Short-Term Memory* (LSTM), yang akan dijelaskan di Subbab 2.8.

2.8 Long Short-Term Memory (LSTM)

Pada Subbab 2.7, RNN diketahui memiliki masalah yang berkaitan dengan *vanishing* dan *exploding gradient*. Ini merupakan masalah umum di RNN yang menyebabkan model menjadi tidak stabil, terutama ketika memproses data dengan ketergantungan jangka panjang karena gradien yang terlalu kecil atau terlalu besar menghambat proses pembelajaran. Untuk mengatasi masalah tersebut, model yang sering digunakan adalah LSTM. LSTM memiliki dua komponen utama, yaitu *cell state* ($\mathbf{c}_t$) dan *hidden state* ($\mathbf{h}_t$), yang berfungsi untuk menyimpan dan memperbarui informasi dalam jangka panjang. Mekanisme ini membantu mengurangi dampak dari masalah *vanishing* dan *exploding gradient*, serta memungkinkan LSTM belajar lebih efektif dibandingkan RNN standar. Berikut adalah ilustrasi sel LSTM ditunjukkan pada Gambar 2.6.



Gambar 2.6: Ilustrasi sel LSTM.

Pada LSTM, kedua komponen tersebut memiliki tiga jenis *gate* yang mengatur aliran informasi, yaitu *input gate* ($\mathbf{i}_t$), *forget gate* ($\mathbf{f}_t$), dan *output gate* ($\mathbf{o}_t$). Berikut adalah penjelasan fungsi ketiga jenis *gate* sebagai berikut:

1. *Input Gate* ($\mathbf{i}_t$) bertujuan untuk menentukan nilai kandidat *cell state* baru ($\tilde{\mathbf{c}}_t$) yang mungkin akan ditambahkan ke dalam *cell state*.
2. *Forget Gate* ($\mathbf{f}_t$) bertujuan untuk menentukan informasi yang harus dihapus dari *cell state*.
3. *Output Gate* ($\mathbf{o}_t$) bertujuan untuk mengendalikan informasi yang harus dilupakan atau dipertahankan pada *cell state*.

Menurut [11, hlm. 293], perhitungan pada LSTM dapat dihitung sebagai berikut:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i),$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f),$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c),$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o),$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

di mana:

- $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_o, \mathbf{W}_c$: bobot (matriks) yang digunakan pada setiap *gate* dan kandidat *cell state* baru.
- $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_c, \mathbf{b}_o$: vektor bias yang digunakan pada setiap *gate* dan kandidat *cell state* baru.
- $\mathbf{x}_t$: vektor *input* pada waktu ke- t .
- $\mathbf{h}_{t-1}$: vektor *hidden state* pada waktu ke- $t - 1$.
- σ : fungsi aktivasi sigmoid.
- $\tanh$: fungsi aktivasi tanh.
- $\odot$: perkalian antar elemen.

Dapat dilihat pada perhitungan LSTM, *cell state* ($\mathbf{c}_t$) diperbarui melalui kombinasi antara informasi sebelumnya ($\mathbf{c}_{t-1}$) yang dikontrol *forget gate* ($\mathbf{f}_t$) dan informasi baru yang ditentukan oleh *input gate* ($\mathbf{i}_t$) serta kandidat *cell state* baru ($\tilde{\mathbf{c}}_t$). Kedua *gate* tersebut menggunakan fungsi aktivasi sigmoid sehingga perubahan yang terjadi pada *cell state* tetap terkendali. Selanjutnya, kandidat *cell state* ($\tilde{\mathbf{c}}_t$) menggunakan fungsi aktivasi tanh agar menjaga saat skala nilai agar tidak menjadi terlalu besar atau kecil saat ditambahkan ke dalam *cell state*. Dengan cara ini, LSTM dapat mencegah masalah *vanishing gradient* dengan mempertahankan informasi yang relevan untuk periode waktu yang lama.

Selain itu, *output gate* ($\mathbf{o}_t$) menggunakan fungsi aktivasi sigmoid untuk mengendalikan seberapa banyak informasi dari *cell state* yang akan digunakan dalam *hidden state* ($\mathbf{h}_t$). Fungsi aktivasi tanh diterapkan pada *cell state* ($\mathbf{c}_t$) agar nilai *hidden state* ($\mathbf{h}_t$) tetap dalam batasan tertentu. Hal ini membantu mencegah *exploding gradient* dengan membatasi besarnya nilai yang dialirkan ke seluruh komponen. Dengan demikian, LSTM mampu mengatasi keterbatasan RNN dalam menangani data dengan lebih stabil dan efisien.

Algoritma 3 LSTM

- 1: **Masukan:** *input* pada waktu ke- t ($\mathbf{x}_t$), *hidden state* sebelumnya ($\mathbf{h}_{t-1}$), *cell state* sebelumnya ($\mathbf{c}_{t-1}$), bobot ($\mathbf{W}$), bias ($\mathbf{b}$), fungsi aktivasi sigmoid (σ), fungsi aktivasi tanh.
 - 2: Untuk $t = 1, \dots, T$ lakukan:
 - 3: $\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i)$
 - 4: $\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f)$
 - 5: $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c)$
 - 6: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$
 - 7: $\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o)$
 - 8: $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$
 - 9: **Keluaran:** *hidden state* ($\mathbf{h}_t$) dan *cell state* ($\mathbf{c}_t$) yang akan digunakan untuk iterasi selanjutnya.
-

2.9 Metrik Evaluasi Model

Metrik evaluasi adalah metrik yang digunakan untuk menilai performa suatu model dalam proses pembelajaran mesin. Metrik ini membantu menentukan seberapa baik model dalam membuat prediksi berdasarkan data yang diberikan. Dalam regresi, metrik ini digunakan untuk menentukan seberapa baik model dalam memprediksi nilai kontinu. Pada skripsi ini, metrik yang digunakan adalah *Mean Squared Error* (MSE). MSE mengukur rata-rata kuadrat selisih antara nilai aktual dan nilai prediksi, oleh karena itu semakin rendah nilai MSE, semakin baik model dalam membuat prediksi. Namun, akibat pengkuadratkan kesalahan pada MSE, metrik ini lebih sensitif terhadap *outlier*, sehingga nilai kesalahan bisa meningkat tajam jika terdapat data ekstrem. Rumus MSE adalah sebagai berikut:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

di mana:

1. n : jumlah data.
2. y_i : nilai sebenarnya dari variabel terikat untuk data ke- i .
3. $\hat{y}_i$: nilai prediksi dari model untuk data ke- i .

BAB 3

IMPLEMENTASI MODEL XGBOOST DAN LSTM

Pada bab ini akan dijelaskan proses pengolahan data hingga pembentukan model. Selain itu, disertakan juga perhitungan manual untuk memberikan gambaran mengenai cara kerja XGBoost dan LSTM.

3.1 Pengumpulan dan Prapengolahan Data

Data dalam skripsi ini berasal dari Profil Kesehatan Indonesia tahun 2018 hingga 2022 [12, 13, 14, 15, 16], yang disusun oleh Kementerian Kesehatan RI dan memuat berbagai indikator kesehatan, termasuk jumlah kasus DBD. Rentang tahun tersebut dipilih karena format datanya konsisten dan tersedia secara resmi. Mulai 2023, penambahan empat provinsi baru di Papua (Papua Pegunungan, Papua Tengah, Papua Selatan, dan Papua Barat Daya) mengubah struktur administratif wilayah, sehingga data setelah 2022 kurang sebanding dengan data sebelumnya.

Pemilihan variabel dalam skripsi ini didasarkan pada faktor-faktor yang memengaruhi penyebaran DBD, mencakup aspek sosiodemografis, lingkungan, serta fasilitas dan pelayanan kesehatan. Variabel sosiodemografis dipertimbangkan karena mencerminkan kondisi sosial ekonomi suatu wilayah yang turut memengaruhi pola penyebaran penyakit. Pemahaman terhadap variabel-variabel ini mendukung pembangunan model prediktif yang lebih akurat. Adapun variabel yang digunakan dalam penelitian ini adalah sebagai berikut:

1. **Tahun:** Periode waktu pengamatan dalam himpunan data dari tahun 2018 hingga 2022.
2. **Provinsi:** Lokasi geografis tempat data dikumpulkan yang mencakup seluruh 34 provinsi di Indonesia.
3. **Area:** Area pengamatan tempat data dikumpulkan yang mencakup 6 area di Indonesia, yaitu **Area Sumatra, Area Jawa, Area Kalimantan, Area Sulawesi dan Maluku, Area Papua**, dan **Area Bali dan Nusa Tenggara**.
4. **Persentase Penduduk Miskin:** Proporsi penduduk suatu provinsi yang hidup di bawah garis kemiskinan dibandingkan dengan total jumlah penduduk di provinsi tersebut.
5. **Indeks Pembangunan Manusia (IPM):** Indikator yang mengukur tingkat pembangunan manusia berdasarkan kesehatan, pendidikan, dan standar hidup layak.
6. **Kepadatan Penduduk per KM:** Rasio jumlah rata-rata penduduk yang tinggal per kilometer persegi di suatu provinsi.

7. **Persentase Kabupaten/Kota Melaksanakan Kebijakan Gerakan Masyarakat Hidup Sehat (GERMAS):** Proporsi kabupaten/kota yang aktif melaksanakan kebijakan yang mendorong perilaku hidup sehat.
8. **Persentase Tempat Pengolahan Pangan yang Memenuhi Syarat Sesuai Standar:** Proporsi tempat pengolahan makanan yang memenuhi standar kebersihan dan keamanan pangan.
9. **Persentase Tempat dan Fasilitas Umum yang Dilakukan Pengawasan Sesuai Standar:** Proporsi fasilitas umum yang diawasi untuk memastikan kepatuhan terhadap standar kesehatan.
10. **Jumlah Kabupaten/Kota yang Melakukan Pelayanan Terpadu (PANDU) di Setiap Puskesmas Menurut Provinsi:** Jumlah kabupaten/kota yang menyediakan layanan terpadu di puskesmas.
11. **Persentase Puskesmas dengan Ketersediaan Vaksin Imunisasi Dasar Lengkap Menurut Provinsi:** Proporsi puskesmas yang memiliki vaksin imunisasi dasar lengkap.
12. **Persentase Kabupaten/Kota Melaksanakan Pembinaan Posyandu Aktif:** Proporsi kabupaten/kota yang aktif membina tempat pelayanan kesehatan masyarakat yang berfokus pada ibu dan anak.
13. **Jumlah Sarana Produksi Bidang Kefarmasian Menurut Provinsi:** Total fasilitas produksi kefarmasian yang terdaftar di provinsi tertentu.
14. **Jumlah Alat Kesehatan Menurut Provinsi:** Total alat kesehatan yang terdaftar di provinsi tertentu.
15. **Rasio Puskesmas per Kecamatan:** Rasio jumlah puskesmas dengan jumlah kecamatan di suatu provinsi.
16. **Jumlah Klinik Pratama Menurut Kepemilikan dan Provinsi:** Total jumlah klinik pratama yang teregistrasi berdasarkan jenis kepemilikan.
17. **Jumlah Klinik Utama Teregistrasi Menurut Kepemilikan dan Provinsi:** Total jumlah klinik pratama dan utama yang teregistrasi berdasarkan jenis kepemilikan dan provinsi.
18. **Jumlah Rumah Sakit Menurut Provinsi:** Total jumlah rumah sakit di suatu provinsi.
19. **Rasio Tempat Tidur Rumah Sakit Menurut Provinsi:** Rasio ketersediaan tempat tidur rumah sakit di suatu provinsi.
20. **Kasus DBD:** Total jumlah kasus DBD di suatu provinsi dan periode waktu tertentu.

Variabel dalam penelitian ini dikelompokkan berdasarkan aspek risiko dan penanganan DBD. Kelompok rentan mencakup **Persentase Penduduk Miskin**, **Indeks Pembangunan Manusia (IPM)**, dan **Kepadatan Penduduk per KM**, yang mencerminkan kondisi sosial ekonomi dan lingkungan padat. Aspek kesadaran pencegahan ditunjukkan oleh **Persentase Kabupaten/Kota**

Melaksanakan Kebijakan Gerakan Masyarakat Hidup Sehat (GERMAS), Persentase Kabupaten/Kota Melaksanakan Pembinaan Posyandu Aktif, dan Persentase Tempat dan Fasilitas Umum yang Dilakukan Pengawasan Sesuai Standar. Kontak dengan nyamuk *Aedes aegypti* diwakili oleh Kepadatan Penduduk per KM, Persentase Tempat dan Fasilitas Umum yang Dilakukan Pengawasan Sesuai Standar, dan Persentase Tempat Pengolahan Pangan yang Memenuhi Syarat Sesuai Standar. Data yang digunakan bersifat longitudinal, di mana pengamatan dilakukan terhadap 34 provinsi yang sama selama rentang waktu lima tahun secara berulang, mencakup 20 variabel setiap tahunnya. Namun, dalam proses pemodelan, data dianggap sebagai *cross-section* untuk menghindari perlakuan khusus seperti yang dibutuhkan oleh data longitudinal.

Tabel 3.1: Contoh data

Tahun	Provinsi	IPM	...	Area	Kasus DBD
2018	Aceh	71,19	...	Sumatra	1.533
2018	Sumatra Utara	71,30	...	Sumatra	5.623
2018	Sumatra Barat	71,95	...	Sumatra	2.203
2018	Riau	72,44	...	Sumatra	918
2018	Jambi	70,65	...	Sumatra	729

3.2 Proses Prapengolahan Data

Sebelum data dapat digunakan dalam model pembelajaran mesin, data perlu diproses untuk memastikan kualitas data. Langkah ini bertujuan untuk mengurangi kesalahan dalam model serta meningkatkan kualitas pada data. Pada data ini, langkah-langkah yang dilakukan dalam pra-pemrosesan adalah sebagai berikut:

1. Menangani *missing values*

Pada tahap ini, kode program dirancang untuk menangani *missing values* agar model dapat berjalan dengan baik dan terhindar dari kesalahan akibat data yang tidak lengkap. Untuk data yang sebagian besar bersifat numerik, nilai hilang biasanya ditangani dengan dua cara: imputasi menggunakan rata-rata atau median atau penghapusan baris/kolom jika jumlah nilai hilang relatif sedikit. Namun, hasil pengecekan menunjukkan bahwa tidak terdapat nilai hilang pada seluruh kolom, sehingga proses imputasi maupun penghapusan tidak diperlukan dalam penelitian ini.

2. *Encoding* variabel kategorikal

Pada tahap ini, variabel kategorikal, seperti label atau kategori, diubah menjadi variabel numerik. Hal ini karena kebanyakan model pembelajaran mesin, terutama regresi dan *neural network* tidak dapat memproses variabel kategorikal dalam bentuk teks. Salah satu teknik yang digunakan adalah *one-hot encoding*, yaitu teknik yang mengubah setiap kategori menjadi kolom biner terpisah. Dalam cara ini, setiap kategori akan dibuat menjadi kolom tersendiri, lalu diberi nilai 1 jika data termasuk dalam kategori tersebut, dan 0 jika tidak. Sebagai contoh, jika terdapat **Area** dengan tiga kategori, yaitu Jawa, Sumatera, dan Kalimantan, maka setelah dilakukan *one-hot encoding*, akan terbentuk tiga kolom baru: Jawa, Sumatera,

dan Kalimantan. Jika suatu data berasal dari Jawa, maka nilai pada Jawa adalah 1, sedangkan dua kolom lainnya bernilai 0. Contoh hasil *one-hot encoding* dapat dilihat pada Tabel 3.2.

Tabel 3.2: Contoh *One-Hot Encoding* pada **Area**

ID	Area	Jawa	Sumatera	Kalimantan
1	Jawa	1	0	0
2	Sumatera	0	1	0
3	Jawa	1	0	0
4	Kalimantan	0	0	1

Dalam skripsi ini, variabel bebas yang perlu melalui tahap ini adalah **Area** dan **Provinsi**, karena keduanya merupakan satu-satunya variabel kategorikal dalam data yang perlu diubah menjadi bentuk numerik.

3. Standardisasi data

Pada tahap ini, dilakukan standardisasi data agar setiap variabel bebas memiliki skala yang seragam. Standardisasi menggunakan *Min-Max Scaling* dengan mengubah data ke dalam rentang $[0, 1]$, dan hanya diterapkan pada LSTM karena model ini sensitif terhadap variabel bebas. Sementara itu, XGBoost tidak memerlukan standardisasi karena pemisahan datanya berdasarkan nilai batas antar variabel bebas. Perhitungan standardisasi variabel ($\mathbf{X}'$) diberikan sebagai berikut:

$$\mathbf{X}' = \frac{\mathbf{X} - X_{\min}}{X_{\max} - X_{\min}}$$

di mana

1. $\mathbf{X}$: nilai asli dari data variabel yang akan distandardisasi.
2. $X_{\min}$: nilai minimum dari data $\mathbf{X}$.
3. $X_{\max}$: nilai maksimum dari data $\mathbf{X}$.
4. $\mathbf{X}'$: nilai yang telah diubah ke dalam rentang $[0, 1]$.

4. Memisahkan data latih dan data uji

Setelah data diproses, data akan dibagi secara acak menjadi data latih dan data uji. Data latih akan digunakan untuk membangun dan melatih model pembelajaran mesin, sementara data uji akan digunakan untuk mengevaluasi kerja model agar memastikan hasil prediksi yang akurat. Data ini akan dipecah 80% data latih dan 20% data uji. Data yang digunakan memiliki jumlah total data sebanyak 170, di mana setelah dibagi menghasilkan 136 data latih dan 34 data uji.

Setelah data dibagi menjadi data latih dan uji, langkah berikutnya adalah membangun dan menerapkan model XGBoost dan LSTM. Proses ini diawali dengan *tuning* parameter menggunakan metode *grid search* pada kedua model, guna menemukan kombinasi parameter terbaik yang mampu menangkap pola dan karakteristik data secara optimal. Rincian parameter yang digunakan akan dijelaskan pada subbab berikutnya.

3.3 Paramater-Parameter XGBoost

Pada XGBoost, model dibangun dengan inisialisasi *XGBRegressor*, yang efisien dalam menangani data tabular melalui pohon keputusan. Parameter utama yang digunakan meliputi:

1. *n_estimators* berfungsi untuk menentukan jumlah pohon keputusan yang dibangun agar dapat membentuk sebuah kombinasi.
2. *max_depth* berfungsi untuk mengatur kedalaman maksimum pohon sehingga dapat mengontrol kompleksitas model dan mencegah *overfitting*.
3. *learning_rate* berfungsi untuk mengatur seberapa cepat model belajar dari kesalahan dengan memperbarui bobot secara bertahap.
4. *random_state* berfungsi untuk menetapkan nilai awal untuk menghasilkan hasil yang konsisten dengan mengontrol pengacakan dalam algoritma.

Selain menggunakan parameter yang telah disebutkan, model dilatih menggunakan fungsi *fit()* pada data latih dan targetnya. Evaluasi performa dilakukan dengan *cross-validation*, yaitu membagi data menjadi beberapa *fold* untuk menguji model secara bergantian.

3.4 Paramater-Parameter LSTM

Pada LSTM, model terlebih dahulu diubah menjadi tiga dimensi (*sample, timestep, feature*) agar LSTM dapat memahami data temporal. Setelah diatur ulang, proses pembangunan model dapat mulai dilakukan. Proses pembangunan LSTM dimulai dengan menginisialisasi model *Sequential*. Model ini memungkinkan menumpuk lapisan secara berurutan, di mana variabel terikat dari lapisan sebelumnya menjadi variabel bebas di lapisan selanjutnya. Lapisan-lapisan pada LSTM meliputi:

1. Lapisan LSTM berfungsi untuk mengenali pola dalam data dari lapisan sebelumnya.
2. Lapisan *Dropout* berfungsi untuk mencegah *overfitting* dengan mengabaikan proporsi tertentu dari neuron secara acak selama pelatihan.
3. Lapisan *Dense* berfungsi untuk memproses dan menyempurnakan variabel terikat hingga menghasilkan prediksi akhir.

Sebelum pelatihan, LSTM dikonfigurasi dengan *optimizer* dan *loss function* untuk mendukung proses pembelajaran. Seperti XGBoost, LSTM dilatih menggunakan fungsi *fit()* secara bertahap terhadap data latih dan targetnya. Setelah pelatihan, performa kedua model dievaluasi menggunakan metrik seperti akurasi, kesalahan prediksi, dan kemampuan mengenali tren kasus DBD. Hasil evaluasi ini menjadi dasar penilaian efektivitas model yang akan dibahas pada bab selanjutnya.

3.5 Perhitungan Manual XGBoost

Misalkan data yang digunakan sebagai contoh perhitungan mencakup 6 **Provinsi**, dengan **Jumlah Kasus DBD** sebagai variabel terikat yang dilambangkan dengan y_i , seperti ditunjukkan pada Tabel 3.3:

Tabel 3.3: Data **IPM**, **Kepadatan Penduduk**, **Persentase Puskesmas**, dan **Jumlah Kasus DBD**

Provinsi	IPM	Kepadatan Penduduk	Persentase Puskesmas	Jumlah Kasus DBD
Riau	73,52	73,00	100,00	2.370
Maluku	68,87	37,81	98,49	317
Nusa Tenggara Timur	68,65	276,00	100,00	2.697
Sulawesi Barat	65,10	80,75	100,00	532
Bangka Belitung	72,24	88,00	100,00	1.887
Sulawesi Tengah	72,23	74,00	50,00	918

Perhitungan Prediksi Awal

Pertama, hitung prediksi awal, yaitu rata-rata kasus DBD yang dilambangkan dengan $F_0(\mathbf{x})$:

$$F_0(\mathbf{x}) = \frac{2.370 + 317 + 2.697 + 532 + 1.887 + 918}{6} = 1.453,5$$

Pada regresi, prediksi awal menggunakan rata-rata target karena meminimalkan nilai *loss*, khususnya jika menggunakan MSE. Selanjutnya, dihitung nilai residu yang ditampilkan pada Tabel 3.4.

Tabel 3.4: Residual prediksi awal **Jumlah Kasus DBD**

Provinsi	Jumlah Kasus DBD (y_i)	Residual ($y_i - F_0$)
Riau	2.370	916,5
Maluku	317	-1.136,5
Nusa Tenggara Timur	2.697	1.243,5
Sulawesi Barat	532	-921,5
Bangka Belitung	1.887	433,5
Sulawesi Tengah	918	-535,5

Perhitungan Gain

Sebelum membuat sebuah pohon keputusan, langkah pertama yang dilakukan adalah menentukan variabel utama yang paling berpengaruh dalam proses pemisahan data agar model berjalan dengan optimal. Salah satu cara untuk menentukan variabel yang berpengaruh adalah menghitung *Gain*. Dalam XGBoost, pemilihan batas pemisahan dilakukan secara otomatis berdasarkan *Gain* tertinggi. Namun, pada perhitungan ini, titik pemisahan dalam pembentukan pohon keputusan ditetapkan masing-masing sebesar 70% untuk **IPM**, 80% untuk **Kepadatan Penduduk**, dan 98% untuk **Persentase Puskesmas**. Dalam perhitungan ini, nilai parameter regularisasi (λ) ditetapkan sebesar 0 untuk memudahkan proses perhitungan. Berikut adalah perhitungan *Gain* pada setiap variabel:

1. IPM

Misalkan variabel utama untuk pohon keputusan adalah **IPM**, dengan aturan sebagai berikut:

1. Jika **IPM** kurang dari 70%, maka residual dihitung dari total residual Maluku, NTT, dan Sulawesi Barat, yaitu $-1.136,5 + 1.243,5 - 921,5 = -814,5$.

2. Jika **IPM** lebih dari 70%, maka residual dihitung dari total residual Riau, Bangka Belitung, dan Sulawesi Tengah, yaitu $916,5 - 535,5 + 433,5 = 814,5$.

Gain dihitung dengan rumus :

$$Gain = \frac{(-814,5)^2}{3} + \frac{(814,5)^2}{3} - \frac{(0)^2}{6} = 442.273,5$$

2. Kepadatan Penduduk

Misalkan variabel utama untuk pohon keputusan adalah **Kepadatan Penduduk**, dengan aturan sebagai berikut:

1. Jika **Kepadatan Penduduk** kurang dari 80%, maka residual dihitung dari total residual Maluku, Riau, dan Sulawesi Tengah, yaitu $916,5 - 535,5 - 1.136,5 = -755,49$.
2. Jika **Kepadatan Penduduk** lebih dari 80%, maka residual dihitung dari total residual Nusa Tenggara Timur, Bangka Belitung, dan Sulawesi Barat, yaitu $1.243,5 + 433,5 - 921,5 = 755,49$.

Gain dihitung dengan rumus :

$$Gain = \frac{(-755,49)^2}{3} + \frac{(755,49)^2}{3} - \frac{(0)^2}{6} = 380.510,09$$

3. Persentase Puskesmas

Misalkan variabel utama untuk pohon keputusan adalah **Persentase Puskesmas**, dengan aturan sebagai berikut:

1. Jika **Persentase Puskesmas** kurang dari 98%, maka residual dihitung dari total residual Sulawesi Tengah, yaitu $-535,5$.
2. Jika **Persentase Puskesmas** lebih dari 98%, maka residual dihitung dari total residual Riau, Maluku, Nusa Tenggara Timur, Bangka Belitung, dan Sulawesi Barat, yaitu $916,5 + (-1.136,5) + 1.243,5 + (-921,5) + 433,5 = 535,5$.

Gain dihitung dengan rumus :

$$Gain = \frac{(-535,5)^2}{1} + \frac{(535,5)^2}{5} - \frac{(0)^2}{6} = 344.112,3$$

Setelah menghitung nilai *Gain* untuk ketiga variabel, variabel dengan kontribusi terbesar dalam pembentukan pohon keputusan adalah **IPM**, diikuti oleh **Kepadatan Penduduk**, dan terakhir **Persentase Puskesmas** yang ditunjukkan pada Tabel 3.5.

Tabel 3.5: *Gain* ketiga variabel

Variabel	<i>Gain</i>
IPM	442.273,50
Kepadatan Penduduk	380.510,09
Persentase Puskesmas	344.112,50

Pohon Pertama

Setelah menghitung *Gain*, pembuatan pohon keputusan dimulai dengan **IPM** sebagai akar karena kontribusinya paling besar. Cabang selanjutnya dipisah berdasarkan **Kepadatan Penduduk**, diikuti oleh **Persentase Puskesmas** sebagai pemisahan akhir. Dengan struktur pohon yang telah ditentukan, langkah berikutnya adalah menghitung bobot prediksi residual di setiap simpul berdasarkan aturan yang telah ditetapkan.

Pemisahan Pertama

Pertama, dilakukan perhitungan bobot prediksi residual untuk variabel pertama, yaitu **IPM**, dengan aturan sebagai berikut:

1. Jika **IPM** kurang dari 70%, maka bobot prediksi residual (w_L) adalah rata-rata residual dari Maluku, Nusa Tenggara Timur, dan Sulawesi Barat. Berikut perhitungan w_L adalah sebagai berikut:

$$-\frac{-1.136,5 + 1.243,5 - 921,5}{3} = -\frac{-814,5}{3} = 271,5$$

2. Jika **IPM** lebih dari 70%, maka bobot prediksi residual (w_R) adalah rata-rata residual dari Riau, Bangka Belitung, dan Sulawesi Tengah. Berikut perhitungan w_R adalah sebagai berikut:

$$-\frac{916,5 - 535,5 + 433,5}{3} = -\frac{814,5}{3} = -271,5$$

Misalkan *learning rate* (β) = 0,1, yang mengatur besarnya kontribusi pohon baru terhadap model. Prediksi baru dihitung dengan rumus $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \beta \cdot w_j$, di mana w_j disesuaikan dengan posisi data setelah pemisahan (w_L untuk cabang kiri dan w_R untuk cabang kanan). Hasil prediksi ditampilkan pada Tabel 3.6.

Tabel 3.6: Prediksi baru setelah pemisahan pertama

Provinsi	IPM	Bobot Prediksi Residual (w_j)	Prediksi Baru (F_1)
Sulawesi Barat	65,10	271,5	$1.453,5 + 0,1(271,5) = 1.480,65$
Nusa Tenggara Timur	68,65	271,5	$1.453,5 + 0,1(271,5) = 1.480,65$
Maluku	68,87	271,5	$1.453,5 + 0,1(271,5) = 1.480,65$
Sulawesi Tengah	72,23	-271,5	$1.453,5 + 0,1(-271,5) = 1.426,35$
Bangka Belitung	72,24	-271,5	$1.453,5 + 0,1(-271,5) = 1.426,35$
Riau	73,52	-271,5	$1.453,5 + 0,1(-271,5) = 1.426,35$

Selanjutnya, perhitungan residual terbaru dilakukan, seperti ditunjukkan pada Tabel 3.7.

Tabel 3.7: Perhitungan residual berdasarkan prediksi pertama (F_1)

Provinsi	Jumlah Kasus DBD (y_i)	Prediksi Baru (F_1)	Residual ($y_i - F_1$)
Sulawesi Barat	532	1.480,65	-948,65
Nusa Tenggara Timur	2.697	1.480,65	1.216,35
Maluku	317	1.480,65	-1.163,65
Sulawesi Tengah	918	1.426,35	-508,35
Bangka Belitung	1.887	1.426,35	460,65
Riau	2.370	1.426,35	943,65

Pemisahan Kedua

Selanjutnya, dilakukan perhitungan bobot prediksi residual untuk variabel kedua, yaitu **Kepadatan Penduduk**, dengan aturan sebagai berikut:

1. Jika **Kepadatan Penduduk** kurang dari 80%, maka bobot prediksi residual (w_L) adalah rata-rata residual dari Maluku, Riau, dan Sulawesi Tengah. Berikut perhitungan w_L adalah sebagai berikut:

$$-\frac{-1.163,65 + 943,65 - 508,35}{3} = \frac{728,35}{3} = 242,78$$

2. Jika **Kepadatan Penduduk** lebih dari 80%, maka bobot prediksi residual (w_R) adalah rata-rata residual dari Nusa Tenggara Timur, Bangka Belitung, dan Sulawesi Barat. Berikut perhitungan w_R adalah sebagai berikut:

$$-\frac{1.216,35 + 460,65 - 948,65}{3} = -\frac{728,35}{3} = -242,78$$

Prediksi baru pada pemisahan kedua dapat dilihat pada Tabel 3.8.

Tabel 3.8: Prediksi baru setelah pemisahan kedua

Provinsi	Kepadatan Penduduk	Bobot Prediksi Residual (w_j)	Prediksi Baru (F_2)
Maluku	37,80	242,78	$1.480,65 + 0,1(242,78) = 1.504,93$
Riau	73,00	242,78	$1.426,35 + 0,1(242,78) = 1.450,63$
Sulawesi Tengah	74,00	242,78	$1.426,35 + 0,1(242,78) = 1.450,63$
Sulawesi Barat	80,75	-242,78	$1.480,65 + 0,1(-242,78) = 1.456,37$
Bangka Belitung	88,00	-242,78	$1.426,35 + 0,1(-242,78) = 1.402,07$
Nusa Tenggara Timur	276,00	-242,78	$1.480,65 + 0,1(-242,78) = 1.456,37$

Selanjutnya, perhitungan residual terbaru, seperti yang ditampilkan pada Tabel 3.9.

Tabel 3.9: Perhitungan residual berdasarkan prediksi kedua (F_2)

Provinsi	Jumlah Kasus DBD (y_i)	Prediksi Baru (F_2)	Residual ($y_i - F_2$)
Maluku	317	1.504,93	-1.187,93
Riau	2.370	1.450,63	919,37
Sulawesi Tengah	918	1.450,63	-532,63
Sulawesi Barat	532	1.456,37	-924,37
Bangka Belitung	1.887	1.402,07	484,93
Nusa Tenggara Timur	2.697	1.456,37	1.240,63

Pemisahan Ketiga

Terakhir, dilakukan perhitungan bobot prediksi residual untuk variabel ketiga, yaitu **Persentase Puskesmas**, dengan aturan sebagai berikut:

1. Jika **Persentase Puskesmas** kurang dari 98%, maka bobot prediksi residual (w_L) adalah rata-rata residual dari Sulawesi Tengah. Berikut perhitungan w_L adalah sebagai berikut:

$$-\frac{-532,63}{1} = 532,63$$

2. Jika **Persentase Puskesmas** lebih dari 98%, maka bobot prediksi residual (w_R) adalah rata-rata residual dari Riau, Maluku, Nusa Tenggara Timur, Bangka Belitung, dan Sulawesi Barat. Berikut perhitungan w_R adalah sebagai berikut:

$$-\frac{919,37 + (-1.187,93) + 1.240,63 + (-924,37) + 484,93}{5} = -\frac{532,63}{5} = -106,53$$

Prediksi baru pada pemisahan ketiga dapat dilihat pada Tabel 3.10.

Tabel 3.10: Prediksi baru setelah pemisahan ketiga

Provinsi	Persentase Puskesmas	Bobot Prediksi Residual (w_j)	Prediksi Baru (F_3)
Sulawesi Tengah	50,00	532,63	$1.450,63 + 0,1(532,63) = 1.503,89$
Maluku	98,49	-106,53	$1.504,93 + 0,1(-106,53) = 1.494,28$
Riau	100,00	-106,53	$1.450,63 + 0,1(-106,53) = 1.439,98$
Nusa Tenggara Timur	100,00	-106,53	$1.456,37 + 0,1(-106,53) = 1.445,72$
Sulawesi Barat	100,00	-106,53	$1.456,37 + 0,1(-106,53) = 1.445,72$
Bangka Belitung	100,00	-106,53	$1.402,07 + 0,1(-106,53) = 1.391,42$

Selanjutnya, perhitungan residual terbaru, seperti yang ditampilkan pada Tabel 3.11.

Tabel 3.11: Perhitungan residual berdasarkan prediksi ketiga (F_3)

Provinsi	Jumlah Kasus DBD (y_i)	Prediksi Baru (F_3)	Residual ($y_i - F_3$)
Sulawesi Tengah	918	1.503,89	-585,89
Maluku	317	1.494,28	-1.177,28
Riau	2.370	1.439,98	930,02
Nusa Tenggara Timur	2.697	1.445,72	1.251,28
Sulawesi Barat	532	1.445,72	-913,72
Bangka Belitung	1.887	1.391,42	495,58

Hasil Akhir XGBoost Setelah Tiga Pemisahan di Pohon Pertama

Setelah tiga pemisahan pada pohon pertama XGBoost, prediksi menunjukkan terdapat perbaikan bertahap. Residual menurun secara bertahap pada beberapa provinsi seperti Sulawesi Barat dan Riau, menandakan bahwa model semakin mendekati nilai aktual. Namun, provinsi seperti Maluku dan Nusa Tenggara Timur masih menunjukkan residual tinggi dan relatif stabil, menandakan bahwa model belum sepenuhnya menangkap pola data dengan baik, seperti ditunjukkan pada Tabel 3.12.

Tabel 3.12: Hasil prediksi dan residual pada setiap pemisahan di pohon pertama

Provinsi	Pemisahan 1		Pemisahan 2		Pemisahan 3	
	Prediksi	Residual	Prediksi	Residual	Prediksi	Residual
Sulawesi Barat	1.480,65	-948,65	1.456,37	-924,37	1.445,72	-913,72
Nusa Tenggara Timur	1.480,65	1.216,35	1.456,37	1.240,63	1.445,72	1.251,28
Maluku	1.480,65	-1.163,65	1.504,93	-1.187,93	1.494,28	-1.177,28
Sulawesi Tengah	1.426,35	-508,35	1.450,63	-532,63	1.503,89	-585,89
Bangka Belitung	1.426,35	460,65	1.402,07	484,93	1.391,42	495,58
Riau	1.426,35	943,65	1.450,63	919,37	1.439,98	930,02

Pohon Kedua

Setelah menghitung nilai prediksi dan residual akhir pada pohon pertama, proses dilanjutkan ke tahap berikutnya dalam XGBoost, yaitu pembangunan pohon kedua. Pada tahap ini, hasil prediksi terakhir yang diperoleh dari pohon pertama tidak hanya menjadi acuan evaluasi, tetapi juga digunakan sebagai dasar pembaruan prediksi untuk iterasi selanjutnya. Berikut adalah prediksi dan residual awal untuk pohon kedua yang ditunjukkan pada Tabel 3.13.

Tabel 3.13: Prediksi dan residual awal pohon kedua

Provinsi	Prediksi Akhir	Residual Akhir
Sulawesi Barat	1.445,72	-913,72
Nusa Tenggara Timur	1.445,72	1.251,28
Maluku	1.494,28	-1.177,28
Sulawesi Tengah	1.503,89	-585,89
Bangka Belitung	1.391,42	495,58
Riau	1.439,98	930,02

Pemisahan Pertama

Dari tabel di atas, pohon kedua dapat dibuat untuk memperbaiki prediksi dengan mengurangi residual dari pohon pertama. Mulai dari simpul akar di **IPM**, dilakukan perhitungan bobot prediksi residual, dengan aturan sebagai berikut:

1. Jika **IPM** kurang dari 70%, maka bobot prediksi residual (w_L) adalah rata-rata residual dari Maluku, Nusa Tenggara Timur, dan Sulawesi Barat. Berikut perhitungan w_L adalah sebagai berikut:

$$-\frac{-1.177,28 + 1.251,28 - 913,72}{3} = \frac{839,72}{3} = 279,9$$

2. Jika **IPM** lebih dari 70%, maka bobot prediksi residual (w_R) adalah rata-rata residual dari Riau, Bangka Belitung, dan Sulawesi Tengah. Berikut perhitungan w_R adalah sebagai berikut:

$$-\frac{930,02 - 585,89 + 495,58}{3} = -\frac{839,71}{3} = -279,9$$

Dengan demikian, prediksi baru pada pemisahan pertama dapat dilihat pada Tabel 3.14.

Tabel 3.14: Prediksi baru setelah pemisahan pertama

Provinsi	IPM	Bobot Prediksi Residual (w_j)	Prediksi Baru (F_1)
Sulawesi Barat	65,10	279,9	$1.445,72 + 0,1(279,9) = 1.473,71$
Nusa Tenggara Timur	68,65	279,9	$1.445,72 + 0,1(279,9) = 1.473,71$
Maluku	68,87	279,9	$1.494,28 + 0,1(279,9) = 1.522,27$
Sulawesi Tengah	72,23	-279,9	$1.503,89 + 0,1(-279,9) = 1.475,90$
Bangka Belitung	72,24	-279,9	$1.391,42 + 0,1(-279,9) = 1.363,43$
Riau	73,52	-279,9	$1.439,98 + 0,1(-279,9) = 1.411,99$

Selanjutnya, perhitungan residual terbaru dilakukan, seperti ditunjukkan pada Tabel 3.15

Tabel 3.15: Perhitungan residual berdasarkan prediksi pertama (F_1)

Provinsi	Jumlah Kasus DBD (y_i)	Prediksi Baru (F_1)	Residual ($y_i - F_1$)
Sulawesi Barat	532	1.473,71	-941,71
Nusa Tenggara Timur	2.697	1.473,71	1.223,29
Maluku	317	1.522,27	-1.205,27
Sulawesi Tengah	918	1.475,90	-557,90
Bangka Belitung	1.887	1.363,43	523,57
Riau	2.370	1.411,99	958,01

Pemisahan Kedua

Selanjutnya, dilakukan perhitungan bobot prediksi residual untuk variabel kedua, yaitu **Kepadatan Penduduk**, dengan aturan sebagai berikut:

1. Jika **Kepadatan Penduduk** kurang dari 80%, maka bobot prediksi residual (w_L) adalah rata-rata residual dari Maluku, Riau, dan Sulawesi Tengah. Berikut perhitungan w_L adalah sebagai berikut:

$$-\frac{-1.205,27 + 958,01 - 557,90}{3} = \frac{805,16}{3} = 268,38$$

2. Jika **Kepadatan Penduduk** lebih dari 80%, maka bobot prediksi residual (w_R) adalah rata-rata residual dari Nusa Tenggara Timur, Bangka Belitung, dan Sulawesi Barat. Berikut perhitungan w_R adalah sebagai berikut:

$$-\frac{1.223,29 + 523,57 - 941,71}{3} = -\frac{805,15}{3} = -268,38$$

Prediksi baru pada pemisahan kedua dapat dilihat pada Tabel 3.16.

Tabel 3.16: Prediksi baru setelah pemisahan kedua

Provinsi	Kepadatan Penduduk	Bobot Prediksi Residual (w_j)	Prediksi Baru (F_2)
Maluku	37,80	268,38	$1.522,27 + 0,1(268,38) = 1.549,11$
Riau	73,00	268,38	$1.411,99 + 0,1(268,38) = 1.438,83$
Sulawesi Tengah	74,00	268,38	$1.475,90 + 0,1(268,38) = 1.502,74$
Sulawesi Barat	80,75	-268,38	$1.473,71 + 0,1(-268,38) = 1.446,87$
Bangka Belitung	88,00	-268,38	$1.363,43 + 0,1(-268,38) = 1.336,59$
Nusa Tenggara Timur	276,00	-268,38	$1.473,71 + 0,1(-268,38) = 1.446,87$

Selanjutnya, perhitungan residual terbaru, seperti yang ditampilkan pada Tabel 3.17.

Tabel 3.17: Perhitungan residual berdasarkan prediksi kedua (F_2)

Provinsi	Jumlah Kasus DBD (y_i)	Prediksi Baru (F_2)	Residual ($y_i - F_2$)
Maluku	317	1.549,11	-1.232,11
Riau	2.370	1.438,83	931,17
Sulawesi Tengah	918	1.502,74	-584,74
Sulawesi Barat	532	1.446,87	-914,87
Bangka Belitung	1.887	1.336,59	550,41
Nusa Tenggara Timur	2.697	1.446,87	1.250,13

Pemisahan Ketiga

Terakhir, dilakukan perhitungan bobot prediksi residual untuk variabel ketiga, yaitu **Persentase Puskesmas**, dengan aturan sebagai berikut:

1. Jika **Persentase Puskesmas** kurang dari 98%, maka bobot prediksi residual (w_L) adalah rata-rata residual dari Sulawesi Tengah. Berikut perhitungan w_L adalah sebagai berikut:

$$-\frac{-584,74}{1} = 584,74$$

2. Jika **Persentase Puskesmas** lebih dari 98%, maka bobot prediksi residual (w_R) adalah rata-rata residual dari Riau, Maluku, Nusa Tenggara Timur, Bangka Belitung, dan Sulawesi Barat. Berikut perhitungan w_R adalah sebagai berikut:

$$-\frac{931,17 + (-1.232,11) + 1.250,13 + (-914,87) + 550,41}{5} = -\frac{584,73}{5} = -116,95$$

Prediksi baru pada pemisahan ketiga dapat dilihat pada Tabel 3.18.

Tabel 3.18: Prediksi baru setelah pemisahan ketiga

Provinsi	Persentase Puskesmas	Bobot Prediksi Residual (w_j)	Prediksi Baru (F_3)
Sulawesi Tengah	50,00	584,74	$1.502,74 + 0,1(584,74) = 1.561,21$
Maluku	98,49	-116,95	$1.549,11 + 0,1(-116,95) = 1.537,41$
Riau	100,00	-116,95	$1.438,83 + 0,1(-116,95) = 1.427,13$
Nusa Tenggara Timur	100,00	-116,95	$1.446,87 + 0,1(-116,95) = 1.435,17$
Sulawesi Barat	100,00	-116,95	$1.446,87 + 0,1(-116,95) = 1.435,17$
Bangka Belitung	100,00	-116,95	$1.336,59 + 0,1(-116,95) = 1.324,89$

Selanjutnya, perhitungan residual terbaru, seperti yang ditampilkan pada Tabel 3.19.

Tabel 3.19: Perhitungan residual berdasarkan prediksi ketiga (F_3)

Provinsi	Jumlah Kasus DBD (y)	Prediksi Baru (F_3)	Residual ($y_i - F_3$)
Sulawesi Tengah	918	1.561,21	-643,21
Maluku	317	1.537,41	-1.220,41
Riau	2.370	1.427,13	942,87
Nusa Tenggara Timur	2.697	1.435,17	1.261,83
Sulawesi Barat	532	1.435,17	-903,17
Bangka Belitung	1.887	1.324,89	562,11

Hasil Akhir XGBoost Setelah Tiga Pemisahan di Pohon Kedua

Setelah tiga pemisahan pada pohon kedua XGBoost, model menunjukkan bahwa beberapa provinsi mengalami perubahan residual yang bervariasi. Beberapa provinsi seperti Sulawesi Barat dan Riau mengalami sedikit perbaikan dengan penurunan residual, menandakan bahwa model semakin mendekati nilai aktual untuk provinsi tersebut. Namun, provinsi lain seperti Maluku dan Sulawesi Tengah mengalami peningkatan residual, yang mengindikasikan bahwa model masih kesulitan menangkap pola data yang kompleks di daerah tersebut. Selain itu, provinsi seperti Bangka

Belitung dan Nusa Tenggara Timur masih memiliki residual yang cukup besar, baik positif maupun negatif, menunjukkan bahwa meskipun model telah diperbarui, kesalahan prediksi masih cukup signifikan, seperti yang ditunjukkan pada Tabel 3.20.

Tabel 3.20: Hasil prediksi dan residual pada setiap pemisahan di pohon kedua

Provinsi	Pemisahan 1		Pemisahan 2		Pemisahan 3	
	Prediksi	Residual	Prediksi	Residual	Prediksi	Residual
Sulawesi Barat	1.473,71	−941,71	1.446,87	−914,87	1.435,17	−903,17
Nusa Tenggara Timur	1.473,71	1.223,29	1.446,87	1.250,13	1.435,17	1.261,83
Maluku	1.522,27	−1.205,27	1.549,11	−1.232,11	1.537,41	−1.220,41
Sulawesi Tengah	1.475,90	−557,90	1.502,74	−584,74	1.561,21	−643,21
Bangka Belitung	1.363,43	523,57	1.336,59	550,41	1.324,89	562,11
Riau	1.411,99	958,01	1.438,83	931,17	1.427,13	942,87

Perbandingan Hasil Antara Pohon Pertama dan Kedua

Berdasarkan perbandingan residual pada pohon pertama dan kedua yang ditunjukkan pada Tabel 3.21, XGBoost memperbarui prediksi secara bertahap untuk meningkatkan akurasi. Namun, pada beberapa provinsi seperti Sulawesi Tengah dan Maluku, residual justru mengalami peningkatan, yang menunjukkan bahwa model masih kesulitan dalam menyesuaikan prediksi dengan nilai aktual. Di Sulawesi Tengah, residual naik dari −585,89 menjadi −643,21, sedangkan di Maluku meningkat dari −1.177,28 menjadi −1.220,41.

Kondisi serupa juga terlihat di Riau dan Nusa Tenggara Timur, di mana residual masing-masing meningkat dari 930,02 ke 942,87, dan dari 1.251,28 ke 1.261,83. Bahkan di Bangka Belitung, residual turut mengalami kenaikan dari 495,58 menjadi 562,11. Kenaikan residual ini menandakan bahwa prediksi model pada beberapa provinsi masih belum optimal dan memerlukan penyesuaian lebih lanjut. Hal ini memperlihatkan bahwa pembaruan pada pohon kedua belum selalu berhasil memberikan perbaikan di semua wilayah.

Sebaliknya, terdapat satu provinsi menunjukkan perbaikan, yaitu Sulawesi Barat yang membaik dari −913,72 menjadi −903,17. Hal ini menunjukkan bahwa meskipun model belum sepenuhnya akurat, pembaruan secara bertahap tetap memberikan dampak positif di beberapa daerah. Secara keseluruhan, pembaruan pada pohon kedua menghasilkan dampak yang bervariasi, sehingga iterasi tambahan masih diperlukan agar model dapat menghasilkan prediksi jumlah kasus DBD yang lebih akurat dan merata di seluruh provinsi.

Tabel 3.21: Residual akhir pada kedua pohon XGBoost

Provinsi	Residual Pohon Pertama	Residual Pohon Kedua
Sulawesi Barat	−913,72	−903,17
Nusa Tenggara Timur	1.251,28	1.261,83
Maluku	−1.177,28	−1.220,41
Sulawesi Tengah	−585,89	−643,21
Bangka Belitung	495,58	562,11
Riau	930,02	942,87

Hasil Prediksi Akhir XGBoost Setelah Kedua Pohon

Setelah dua pohon dibangun dalam perhitungan manual XGBoost, residual masing-masing pohon digunakan untuk menghitung prediksi akhir. Nilai prediksi dari kedua pohon kemudian dievaluasi menggunakan MSE berdasarkan residual di setiap provinsi, dengan perhitungan sebagai berikut:

1. Pohon Pertama

$$\begin{aligned} \text{MSE} &= \frac{(-913,72)^2 + (1.251,28)^2 + (-1.177,28)^2 + \dots + (930,02)^2}{6} \\ &= 729.116,23 \end{aligned}$$

2. Pohon Kedua

$$\begin{aligned} \text{MSE} &= \frac{(-903,17)^2 + (1.261,83)^2 + (-1.220,41)^2 + \dots + (942,87)^2}{6} \\ &= 919.337,03 \end{aligned}$$

Dari hasil perhitungan MSE yang diperoleh, terlihat bahwa MSE pada pohon pertama lebih kecil, yaitu 729.116,23, dibandingkan dengan MSE pada pohon kedua yang sebesar 919.337,03 yang menunjukkan bahwa pohon pertama menghasilkan prediksi yang lebih mendekati nilai aktual dibandingkan dengan pohon kedua.

Misalkan ditambahkan **Provinsi** lain dengan nilai **IPM** sebesar 72%, Kepadatan Penduduk sebesar 85%, dan Persentase Puskesmas sebesar 99%, prediksi jumlah kasus DBD dapat dihitung menggunakan kedua pohon keputusan yang telah dibangun sebelumnya. Berikut adalah hasil perhitungan prediksi untuk **Provinsi** baru.

1. Pohon Pertama

Perhitungan dimulai dari nilai prediksi awal, kemudian bertahap ditambahkan kontribusi masing-masing variabel. Berikut adalah perhitungan prediksi pada pohon pertama:

1. **IPM:** $1.453,5 + 0,1 \cdot (-271,5) = 1.426,35$
2. **Kepadatan Penduduk:** $1.426,35 + 0,1 \cdot (-242,78) = 1.402,07$
3. **Persentase Puskesmas:** $1.402,07 + 0,1 \cdot (-106,53) = 1.391,42$

2. Pohon Kedua

Hasil akhir pada pohon pertama menjadi prediksi awal di pohon kedua. Berikut adalah perhitungan prediksi pada pohon kedua:

1. **IPM:** $1.391,42 + 0,1 \cdot (-279,9) = 1.363,43$
2. **Kepadatan Penduduk:** $1.363,43 + 0,1 \cdot (-268,38) = 1.336,59$
3. **Persentase Puskesmas:** $1.336,59 + 0,1 \cdot (-116,95) = 1.324,89$

Setelah menghitung prediksi dari kedua pohon, rata-rata dari kedua hasil prediksi dapat digunakan untuk memperoleh nilai prediksi akhir bagi provinsi baru. Jadi, prediksi **Jumlah Kasus DBD** untuk **Provinsi** baru adalah sekitar 1.358 kasus.

Untuk mengukur seberapa besar kontribusi masing-masing variabel bebas dalam membentuk pohon keputusan, digunakan nilai *Gain* sebagai ukuran kontribusi. Nilai *Gain* yang tercantum pada Tabel 3.5 digunakan untuk menghitung persentase kontribusi dengan membagi nilai *Gain* tiap variabel dengan total *Gain* seluruh variabel. Berikut adalah perhitungan kontribusi variabel disajikan pada pohon keputusan.

1. **IPM:** $\frac{442.273,50}{1.166.896,09} = 0,3791$
2. **Kepadatan Penduduk:** $\frac{380.510,09}{1.166.896,09} = 0,3261$
3. **Persentase Puskesmas:** $\frac{344.112,50}{1.166.896,09} = 0,2948$

Berdasarkan hasil perhitungan kontribusi variabel, **IPM** memberikan kontribusi terbesar sebesar 0,3791, diikuti oleh **Kepadatan Penduduk** dan **Persentase Puskesmas** masing-masing sebesar 0,3261 dan 0,2948.

3.6 Perhitungan Manual LSTM

Misalkan data yang digunakan adalah data yang sudah digunakan di Subbab 3.5 tanpa menggunakan **Jumlah Kasus DBD** sebagai variabel terikat.

Standardisasi Data

Sebelum data digunakan dalam model, variabel data perlu distandardisasi untuk memastikan bahwa setiap variabel memiliki skala yang seragam. Standardisasi dimulai dengan menghitung nilai *Min* dan *Max* untuk setiap variabel, seperti yang ditunjukkan pada Tabel 3.22.

Tabel 3.22: Nilai *Min* dan *Max* masing-masing variabel

Variabel	<i>Min</i>	<i>Max</i>
IPM	65,10	73,52
Kepadatan Penduduk	37,81	276,00
Persentase Puskesmas	50,00	100,00

Setelah menentukan nilai *Min* dan *Max* pada masing-masing variabel, maka dapat dilakukan perhitungan standardisasi variabel yang dirangkum Tabel 3.23, Tabel 3.24, dan Tabel 3.25.

1. **IPM'**

Tabel 3.23: Nilai standardisasi **IPM**

Provinsi	IPM	IPM'
Riau	73,52	$\frac{73,52-65,10}{73,52-65,10} = 1,00$
Maluku	68,87	$\frac{68,87-65,10}{73,52-65,10} = 0,48$
Nusa Tenggara Timur	68,65	$\frac{68,65-65,10}{73,52-65,10} = 0,45$
Sulawesi Barat	65,10	$\frac{65,10-65,10}{73,52-65,10} = 0,00$
Bangka Belitung	72,24	$\frac{72,24-65,10}{73,52-65,10} = 0,85$
Sulawesi Tengah	72,23	$\frac{72,23-65,10}{73,52-65,10} = 0,85$

2. Kepadatan Penduduk'

Tabel 3.24: Nilai standardisasi **Kepadatan Penduduk**

Provinsi	Kepadatan Penduduk	Kepadatan Penduduk'
Riau	73,00	$\frac{73,00-37,81}{276,00-37,81} = 0,15$
Maluku	37,81	$\frac{37,81-37,81}{276,00-37,81} = 0,00$
Nusa Tenggara Timur	276,00	$\frac{276,00-37,81}{276,00-37,81} = 1,00$
Sulawesi Barat	80,75	$\frac{80,75-37,81}{276,00-37,81} = 0,18$
Bangka Belitung	88,00	$\frac{88,00-37,81}{276,00-37,81} = 0,20$
Sulawesi Tengah	74,00	$\frac{74,00-37,81}{276,00-37,81} = 0,15$

3. Persentase Puskesmas'

Tabel 3.25: Nilai standardisasi **Persentase Puskesmas**

Provinsi	Persentase Puskesmas	Persentase Puskesmas'
Riau	100,00	$\frac{100,00-50,00}{100,00-50,00} = 1,00$
Maluku	98,49	$\frac{98,49-50,00}{100,00-50,00} = 0,97$
Nusa Tenggara Timur	100,00	$\frac{100,00-50,00}{100,00-50,00} = 1,00$
Sulawesi Barat	100,00	$\frac{100,00-50,00}{100,00-50,00} = 1,00$
Bangka Belitung	100,00	$\frac{100,00-50,00}{100,00-50,00} = 1,00$
Sulawesi Tengah	50,00	$\frac{50,00-50,00}{100,00-50,00} = 0,00$

Setelah dilakukan standardisasi pada setiap variabel, seluruh hasil perhitungan standardisasi dirangkum menjadi satu dan disajikan pada Tabel 3.26.

Tabel 3.26: Data standardisasi **IPM**, **Kepadatan Penduduk**, dan **Persentase Puskesmas**

Provinsi	IPM'	Kepadatan Penduduk'	Persentase Puskesmas'
Riau	1,00	0,15	1,00
Maluku	0,48	0,00	0,97
Nusa Tenggara Timur	0,45	1,00	1,00
Sulawesi Barat	0,00	0,18	1,00
Bangka Belitung	0,85	0,20	1,00
Sulawesi Tengah	0,85	0,15	0,00

Inisialisasi Bobot dan Bias

Dalam perhitungan manual LSTM, bobot dan bias harus diinisialisasi terlebih dahulu secara acak. Karena pada perhitungan ini menggunakan 3 variabel bebas, maka jumlah unit LSTM disesuaikan dengan jumlah variabel bebas, sehingga ukuran matriks bobot menjadi 3×3 . Misalkan bobot untuk setiap *gate* diberikan sebagai berikut:

1. Bobot *Forget Gate*

$$\mathbf{W}_f = \begin{bmatrix} 0,40 & 0,30 & -0,20 \\ 0,10 & -0,50 & 0,30 \\ -0,30 & 0,20 & 0,60 \end{bmatrix}$$

2. Bobot *Input Gate*

$$\mathbf{W}_i = \begin{bmatrix} 0,20 & -0,10 & 0,40 \\ 0,30 & 0,60 & -0,20 \\ -0,50 & 0,20 & 0,10 \end{bmatrix}$$

3. Bobot *Candidate Cell State*

$$\mathbf{W}_c = \begin{bmatrix} 0,50 & -0,30 & 0,20 \\ 0,10 & 0,70 & -0,40 \\ -0,20 & 0,30 & 0,50 \end{bmatrix}$$

4. Bobot *Output Gate*

$$\mathbf{W}_o = \begin{bmatrix} 0,30 & -0,20 & 0,50 \\ -0,10 & 0,40 & 0,30 \\ 0,60 & -0,50 & 0,20 \end{bmatrix}$$

Untuk kemudahan perhitungan, bias pada setiap *gate* bernilai 0 ($\mathbf{b}_f = \mathbf{b}_i = \mathbf{b}_c = \mathbf{b}_o = 0$).

Iterasi Pertama

Misalkan data dari provinsi Riau setelah standardisasi ($\mathbf{x}_1$) digunakan sebagai variabel bebas ke LSTM, dan *hidden state* sebelumnya $\mathbf{h}_0$ adalah 0. Perhitungan *forget gate* dilakukan dengan mengalikan bobot matriks $\mathbf{W}_f$ dengan *input* saat ini $\mathbf{x}_1$, kemudian hasilnya diproses menggunakan fungsi sigmoid (σ) untuk mendapatkan nilai antara 0 dan 1. Perhitungan *forget gate* dilakukan sebagai berikut:

$$\begin{aligned} \mathbf{f}_1 &= \sigma(\mathbf{W}_f \cdot \mathbf{x}_1) \\ &= \begin{bmatrix} 0,561 \\ 0,580 \\ 0,581 \end{bmatrix} \end{aligned}$$

Proses perhitungan *input gate* serupa dengan *forget gate*, yaitu dengan mengalikan bobot matriks $\mathbf{W}_i$ dengan *input* saat ini $\mathbf{x}_1$, kemudian hasilnya diproses menggunakan fungsi sigmoid (σ) untuk

menghasilkan nilai antara 0 dan 1. Perhitungan *input gate* dilakukan sebagai berikut:

$$\begin{aligned}\mathbf{i}_1 &= \sigma(\mathbf{W}_i \cdot \mathbf{x}_1) \\ &= \begin{bmatrix} 0,642 \\ 0,547 \\ 0,408 \end{bmatrix}\end{aligned}$$

Perhitungan *candidate cell state* dilakukan dengan mengalikan bobot matriks $\mathbf{W}_c$ dengan *input* saat ini $\mathbf{x}_1$, kemudian hasilnya diproses menggunakan fungsi tanh untuk mendapatkan nilai antara -1 dan 1 . Perhitungan *candidate cell state* dilakukan sebagai berikut:

$$\begin{aligned}\tilde{\mathbf{c}}_1 &= \tanh(\mathbf{W}_c \cdot \mathbf{x}_1) \\ &= \begin{bmatrix} 0,576 \\ -0,193 \\ 0,332 \end{bmatrix}\end{aligned}$$

Cell state baru merupakan hasil gabungan antara *forget gate* dan *input gate* dengan *candidate cell state*. Karena ini langkah pertama, maka $\mathbf{c}_0 = 0$. Perhitungan *cell state* baru dilakukan sebagai berikut:

$$\begin{aligned}\mathbf{c}_1 &= \mathbf{f}_1 \odot \mathbf{c}_0 + \mathbf{i}_1 \odot \tilde{\mathbf{c}}_1 \\ &= \begin{bmatrix} 0,370 \\ -0,106 \\ 0,135 \end{bmatrix}\end{aligned}$$

Perhitungan *output gate* dilakukan dengan mengalikan bobot matriks $\mathbf{W}_o$ dengan *input* saat ini $\mathbf{x}_1$, kemudian hasilnya diproses menggunakan fungsi sigmoid (σ) untuk menghasilkan nilai antara 0 dan 1. Perhitungan *output gate* dilakukan sebagai berikut:

$$\begin{aligned}\mathbf{o}_1 &= \sigma(\mathbf{W}_o \cdot \mathbf{x}_1) \\ &= \begin{bmatrix} 0,683 \\ 0,564 \\ 0,673 \end{bmatrix}\end{aligned}$$

Perhitungan *hidden state* merupakan hasil kombinasi *output gate* dan *cell state* yang sudah melalui fungsi tanh. Jadi, nilai $\mathbf{c}_1$ yang sudah dibentuk tadi diproses dengan tanh supaya nilai berada di rentang -1 sampai 1 , lalu dikalikan dengan *output gate*. Perhitungan *hidden state* dilakukan sebagai berikut:

$$\begin{aligned}\mathbf{h}_1 &= \mathbf{o}_1 \odot \tanh(\mathbf{c}_1) \\ &= \begin{bmatrix} 0,242 \\ -0,060 \\ 0,090 \end{bmatrix}\end{aligned}$$

Iterasi Kedua

Setelah mendapatkan perhitungan dari iterasi pertama, maka data awal untuk iterasi kedua adalah sebagai berikut:

1. *Hidden State* Iterasi Pertama

$$\mathbf{h}_1 = \begin{bmatrix} 0,242 \\ -0,060 \\ 0,090 \end{bmatrix}$$

2. *Cell State* Iterasi Pertama

$$\mathbf{c}_1 = \begin{bmatrix} 0,370 \\ -0,106 \\ 0,135 \end{bmatrix}$$

3. *Input* ke Model untuk Iterasi Kedua

Karena data Riau tidak memiliki informasi lengkap untuk tahun-tahun berikutnya, maka pada iterasi kedua dan seterusnya, model memerlukan *input* tambahan agar proses prediksi dapat dilanjutkan. Dalam hal ini, digunakan *input* yang ditentukan secara acak sebagai simulasi kondisi *input* baru. Misalkan, pada iterasi kedua, *input* yang digunakan dalam model dinyatakan sebagai berikut:

$$\mathbf{x}_2 = \begin{bmatrix} 0,90 \\ 0,20 \\ 0,80 \end{bmatrix}$$

Input ini kemudian diproses kembali melalui komponen-komponen LSTM dengan menggunakan perhitungan yang telah dijelaskan sebelumnya untuk menghasilkan prediksi jumlah kasus tahun selanjutnya.

Perhitungan *forget gate* dilakukan sebagai berikut:

$$\begin{aligned} \mathbf{f}_2 &= \sigma(\mathbf{W}_f \cdot \mathbf{x}_2 + \mathbf{W}_f \cdot \mathbf{h}_1) \\ &= \begin{bmatrix} 0,5795 \\ 0,5772 \\ 0,5546 \end{bmatrix} \end{aligned}$$

Perhitungan *input gate* dilakukan sebagai berikut:

$$\begin{aligned} \mathbf{i}_2 &= \sigma(\mathbf{W}_i \cdot \mathbf{x}_2 + \mathbf{W}_i \cdot \mathbf{h}_1) \\ &= \begin{bmatrix} 0,6388 \\ 0,5618 \\ 0,3884 \end{bmatrix} \end{aligned}$$

Perhitungan *candidate cell state* dilakukan sebagai berikut:

$$\tilde{\mathbf{c}}_2 = \tanh(\mathbf{W}_c \cdot \mathbf{x}_2 + \mathbf{W}_c \cdot \mathbf{h}_1)$$

$$= \begin{bmatrix} 0,6092 \\ -0,1428 \\ 0,2529 \end{bmatrix}$$

Perhitungan *cell state* baru dilakukan sebagai berikut:

$$\begin{aligned} \mathbf{c}_2 &= \mathbf{f}_2 \odot \mathbf{c}_1 + \mathbf{i}_2 \odot \tilde{\mathbf{c}}_2 \\ &= \begin{bmatrix} 0,6038 \\ -0,1414 \\ 0,1731 \end{bmatrix} \end{aligned}$$

Perhitungan *output gate* dilakukan sebagai berikut:

$$\begin{aligned} \mathbf{o}_2 &= \sigma(\mathbf{W}_o \cdot \mathbf{x}_2 + \mathbf{W}_o \cdot \mathbf{h}_1) \\ &= \begin{bmatrix} 0,681 \\ 0,552 \\ 0,688 \end{bmatrix} \end{aligned}$$

Perhitungan *hidden state* dilakukan sebagai berikut:

$$\begin{aligned} \mathbf{h}_2 &= \mathbf{o}_2 \odot \tanh(\mathbf{c}_2) \\ &= \begin{bmatrix} 0,3672 \\ -0,0775 \\ 0,1180 \end{bmatrix} \end{aligned}$$

Iterasi Ketiga

Setelah mendapatkan perhitungan dari iterasi kedua, maka data awal untuk iterasi ketiga adalah sebagai berikut:

1. *Hidden State* Iterasi Kedua

$$\mathbf{h}_2 = \begin{bmatrix} 0,3672 \\ -0,0775 \\ 0,1180 \end{bmatrix}$$

2. *Cell State* Iterasi Kedua

$$\mathbf{c}_2 = \begin{bmatrix} 0,6038 \\ -0,1414 \\ 0,1731 \end{bmatrix}$$

3. *Input* ke Model untuk Iterasi Ketiga

$$\mathbf{x}_3 = \begin{bmatrix} 0,70 \\ 0,50 \\ 0,30 \end{bmatrix}$$

Perhitungan *forget gate* dilakukan sebagai berikut:

$$\begin{aligned}\mathbf{f}_3 &= \sigma(\mathbf{W}_f \cdot \mathbf{x}_3 + \mathbf{W}_f \cdot \mathbf{h}_2) \\ &= \begin{bmatrix} 0,6154 \\ 0,5052 \\ 0,5038 \end{bmatrix}\end{aligned}$$

Perhitungan *input gate* dilakukan sebagai berikut:

$$\begin{aligned}\mathbf{i}_3 &= \sigma(\mathbf{W}_i \cdot \mathbf{x}_3 + \mathbf{W}_i \cdot \mathbf{h}_2) \\ &= \begin{bmatrix} 0,5838 \\ 0,6201 \\ 0,3996 \end{bmatrix}\end{aligned}$$

Perhitungan *candidate cell state* dilakukan sebagai berikut:

$$\begin{aligned}\tilde{\mathbf{c}}_3 &= \tanh(\mathbf{W}_c \cdot \mathbf{x}_3 + \mathbf{W}_c \cdot \mathbf{h}_2) \\ &= \begin{bmatrix} 0,4545 \\ 0,2311 \\ 0,1218 \end{bmatrix}\end{aligned}$$

Perhitungan *cell state* baru dilakukan sebagai berikut:

$$\begin{aligned}\mathbf{c}_3 &= \mathbf{f}_3 \odot \mathbf{c}_2 + \mathbf{i}_3 \odot \tilde{\mathbf{c}}_3 \\ &= \begin{bmatrix} 0,7610 \\ -0,1517 \\ 0,1854 \end{bmatrix}\end{aligned}$$

Perhitungan *output gate* dilakukan sebagai berikut:

$$\begin{aligned}\mathbf{o}_3 &= \sigma(\mathbf{W}_o \cdot \mathbf{x}_3 + \mathbf{W}_o \cdot \mathbf{h}_2) \\ &= \begin{bmatrix} 0,6092 \\ 0,5468 \\ 0,6253 \end{bmatrix}\end{aligned}$$

Perhitungan *hidden state* dilakukan sebagai berikut:

$$\begin{aligned}\mathbf{h}_3 &= \mathbf{o}_3 \odot \tanh(\mathbf{c}_3) \\ &= \begin{bmatrix} 0,3909 \\ -0,0824 \\ 0,1146 \end{bmatrix}\end{aligned}$$

Hasil Prediksi Akhir LSTM Setelah Tiga Iterasi

Setelah melalui tiga iterasi dalam perhitungan manual LSTM untuk memprediksi jumlah kasus DBD, nilai *hidden state* ($\mathbf{h}_t$) yang dihasilkan pada setiap iterasi digunakan untuk menghasilkan prediksi akhir. Prediksi akhir tersebut dilambangkan sebagai $\hat{y}_t$, yang dihitung dengan mengalikan *hidden state* dengan matriks bobot *output* ($\mathbf{W}_y$), dan kemudian ditambahkan dengan bias *output* ($\mathbf{b}_y$). Rumus matematis yang digunakan dalam proses ini ditunjukkan sebagai berikut:

$$\hat{y}_t = \mathbf{W}_y \cdot \mathbf{h}_t + \mathbf{b}_y$$

dimana $\mathbf{W}_y$ berfungsi sebagai matriks bobot output. Untuk kemudahan proses perhitungan manual, nilai bias $\mathbf{b}_y$ diasumsikan bernilai nol. Dengan demikian, prediksi akhir hanya bergantung pada nilai *hidden state* dan bobot *output* yang telah ditentukan sebelumnya.

Hidden state untuk tiga iterasi berturut-turut adalah sebagai berikut:

$$\mathbf{h}_1 = \begin{bmatrix} 0,242 \\ -0,060 \\ 0,090 \end{bmatrix}, \quad \mathbf{h}_2 = \begin{bmatrix} 0,3672 \\ -0,0775 \\ 0,1180 \end{bmatrix}, \quad \mathbf{h}_3 = \begin{bmatrix} 0,3909 \\ -0,0824 \\ 0,1146 \end{bmatrix}$$

Misalkan matriks bobot *output* $\mathbf{W}_y$ yang digunakan dalam proses prediksi adalah:

$$\mathbf{W}_y = \begin{bmatrix} 0,80 & -0,60 & 0,40 \end{bmatrix}$$

Dengan demikian, hasil perhitungan prediksi akhir untuk setiap iterasi dilakukan sebagai berikut:

$$\begin{aligned} \text{Iterasi 1: } \hat{y}_1 &= \begin{bmatrix} 0,80 & -0,60 & 0,40 \end{bmatrix} \cdot \begin{bmatrix} 0,242 \\ -0,060 \\ 0,090 \end{bmatrix} \\ &= 0,2656 \end{aligned}$$

$$\begin{aligned} \text{Iterasi 2: } \hat{y}_2 &= \begin{bmatrix} 0,80 & -0,60 & 0,40 \end{bmatrix} \cdot \begin{bmatrix} 0,3672 \\ -0,0775 \\ 0,1180 \end{bmatrix} \\ &= 0,3875 \end{aligned}$$

$$\begin{aligned} \text{Iterasi 3: } \hat{y}_3 &= \begin{bmatrix} 0,80 & -0,60 & 0,40 \end{bmatrix} \cdot \begin{bmatrix} 0,3909 \\ -0,0824 \\ 0,1146 \end{bmatrix} \\ &= 0,4080 \end{aligned}$$

Berdasarkan hasil perhitungan iterasi pada LSTM untuk memprediksi jumlah kasus DBD, nilai prediksi akhir $\hat{y}_t$ pada setiap iterasi diperoleh sebagai berikut:

$$\hat{y}_1 = 0,2656, \quad \hat{y}_2 = 0,38746, \quad \hat{y}_3 = 0,40800$$

Dari hasil nilai prediksi akhir pada setiap iterasi tersebut, dapat disimpulkan bahwa terdapat

peningkatan nilai secara bertahap pada setiap langkah iterasi. Peningkatan ini mengindikasikan bahwa LSTM secara progresif mampu mempelajari dan menyesuaikan diri dengan pola data historis yang ada, sehingga menghasilkan estimasi prediksi yang semakin mendekati nilai sebenarnya. Hal ini menunjukkan bahwa LSTM dapat mengingat dan belajar dari data sebelumnya dengan baik, sehingga bisa mengenali pola waktu pada data kasus DBD. Karena itu, setiap kali model melakukan perhitungan ulang, prediksi yang dihasilkan menjadi semakin akurat. Dengan kata lain, proses iterasi ini memperlihatkan bagaimana model secara bertahap memperbaiki prediksi dan menjadi lebih tepat seiring waktu berjalan.

Pada iterasi ketiga, nilai prediksi akhir yang dihasilkan oleh LSTM dalam bentuk data yang telah distandardisasi adalah $\hat{y}_3 = 0,40800$. Untuk mengubah nilai prediksi ini kembali ke skala asli, digunakan nilai *Min* dan nilai *Max* dari data asli. Diketahui bahwa nilai *Min* pada **Jumlah Kasus DBD** adalah 317, sedangkan nilai *Max* adalah 2.697. Transformasi dari nilai prediksi yang telah distandardisasi ke nilai asli dilakukan dengan rumus berikut:

$$\begin{aligned} y_3 &= \hat{y}_3 \cdot (Max - Min) + Min \\ &= 1.289,24 \end{aligned}$$

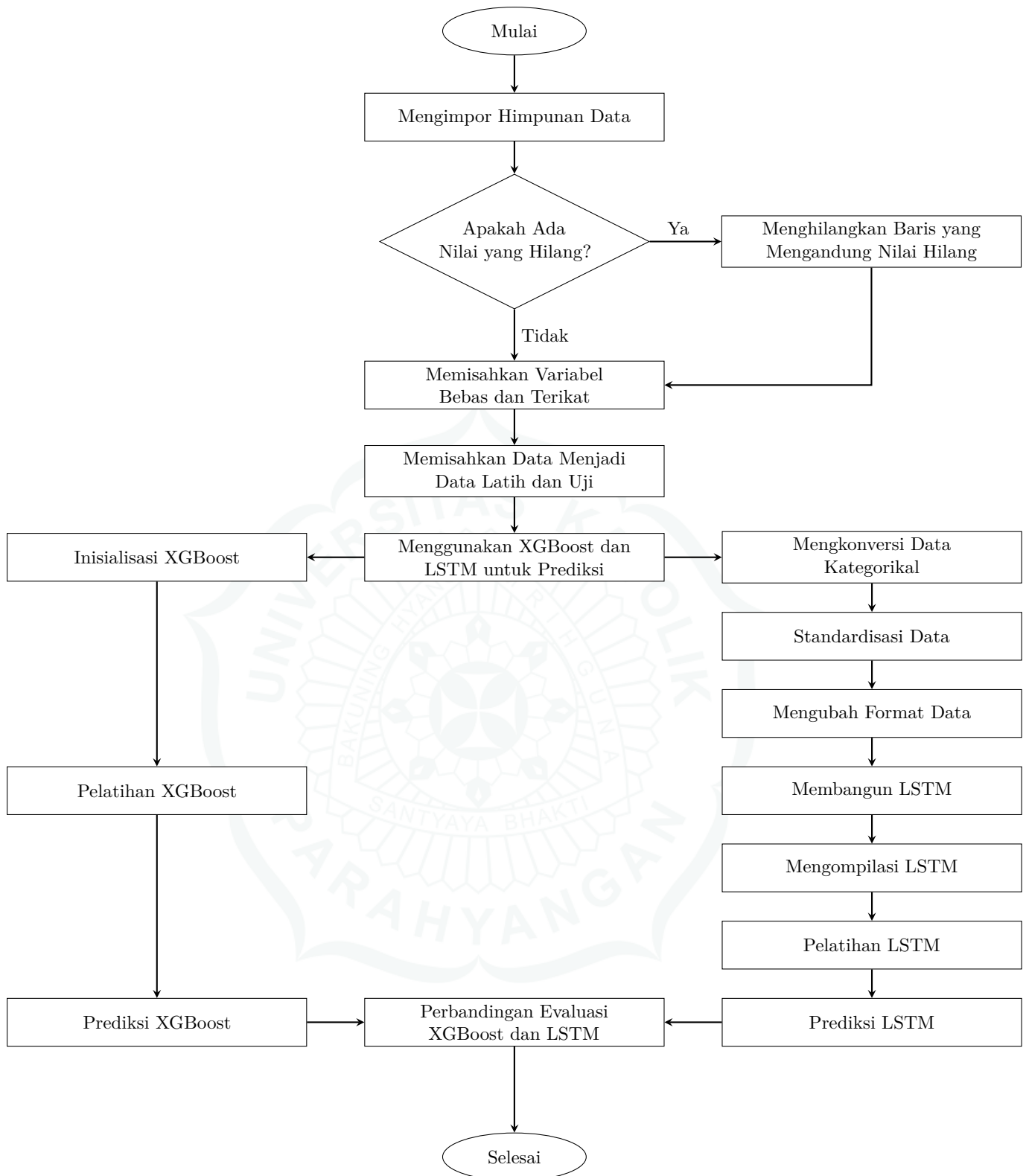
Dengan demikian, hasil prediksi jumlah kasus DBD pada iterasi ketiga adalah sekitar 1.289 kasus. Angka ini menunjukkan estimasi jumlah kasus DBD dalam skala sebenarnya, yang dihasilkan oleh LSTM setelah mempelajari pola data selama tiga iterasi. Prediksi ini mencerminkan kemampuan model dalam memberikan gambaran seberapa besar potensi penyebaran kasus DBD berdasarkan pola historis yang telah dipelajari model.

3.7 Diagram Alir Pemodelan Data XGBoost dan LSTM

Dalam penelitian pemodelan data, visualisasi alur kerja sangat penting untuk memperjelas tahapan analisis yang dilakukan. Diagram alir menjadi alat bantu yang efektif untuk menggambarkan proses kompleks secara ringkas, terutama saat melibatkan dua pendekatan seperti XGBoost dan LSTM. Visualisasi ini membantu menyatukan pemahaman peneliti dan pembaca terhadap struktur pemodelan yang digunakan. Adapun fungsi utama dari diagram alir dalam konteks ini adalah sebagai berikut:

1. Menjelaskan alur kerja XGBoost dan LSTM secara paralel dari tahap input hingga prediksi akhir.
2. Mempermudah pemahaman hubungan antar tahapan dalam proses pemodelan data.
3. Membantu mengidentifikasi bagian yang dapat dioptimalkan untuk meningkatkan kinerja model.
4. Menyediakan kerangka referensi yang sistematis bagi pengembangan atau replikasi model oleh peneliti lain.

Berikut adalah diagram alir pemodelan data menggunakan XGBoost dan LSTM yang ditunjukkan pada Gambar 3.1.



Gambar 3.1: Diagram alir pemodelan data menggunakan XGBoost dan LSTM.

BAB 4

HASIL DAN PEMBAHASAN

Pada bab ini akan dibahas mengenai hasil pemodelan dengan XGBoost dan LSTM beserta analisisnya pada data kasus DBD. Sebelum pemodelan dimulai, data dibagi menjadi dua bagian, yaitu 80% data latih dan 20% data uji. Untuk data latih, sebagian di antaranya digunakan sebagai data validasi guna mengevaluasi performa model selama pelatihan dan menentukan versi terbaik dari XGBoost dan LSTM yang tidak *overfitting*. Untuk memperjelas performa model, ditampilkan tabel nilai kontribusi variabel dari XGBoost dan grafik perbandingan MSE *loss* pada data pelatihan serta validasi dari LSTM.

4.1 Hasil Analisis Menggunakan XGBoost

Pada XGBoost, nilai terbaik dari MSE ditentukan berdasarkan kombinasi parameter yang ditentukan, seperti *n_estimators*, *max_depth*, *learning_rate*, dan *random_state*. Selain itu, *cross-validation* (cv) digunakan agar mendapatkan nilai-nilai parameter yang membuat XGBoost memiliki performa yang terbaik. Dalam skripsi ini digunakan variasi jumlah *fold*, yaitu 3, 5, dan 10. Penggunaan cv yang kecil seperti 3 cenderung lebih cepat dalam proses pelatihan, namun dapat menghasilkan estimasi performa yang kurang stabil. Sebaliknya, cv yang lebih besar seperti 10 memberikan evaluasi yang lebih andal meskipun memerlukan waktu komputasi yang lebih lama. Setelah pelatihan dilakukan menggunakan 125 kombinasi parameter dari Tabel 4.1, diperoleh tiga model dengan nilai MSE terbaik. Hasil terbaik dari masing-masing variasi *cross-validation* disajikan pada Tabel 4.2.

Tabel 4.1: Kombinasi parameter yang digunakan dalam pelatihan XGBoost

Parameter	Nilai yang Diuji
<i>n_estimators</i>	{50, 100, 150, 200, 250}
<i>learning_rate</i>	{0,01, 0,05, 0,10, 0,15, 0,20}
<i>max_depth</i>	{3, 4, 6, 8, 9}
<i>random_state</i>	{42}

Tabel 4.2: Hasil terbaik XGBoost pada berbagai variasi *cross-validation*

cv	<i>learning_rate</i>	<i>max_depth</i>	<i>n_estimators</i>	MSE
3-fold	0,05	3	100	11.848.894,00
5-fold	0,10	3	250	10.812.922,00
10-fold	0,05	3	250	9.016.559,00

Berdasarkan Tabel 4.2, nilai MSE terbaik sebesar 9.016.559,00 diperoleh melalui pencarian parameter optimal dan pengujian menggunakan *cross-validation* 10-fold. Setelah menemukan nilai

terbaik XGBoost, model tersebut perlu diuji kembali pada data uji yang akan ditunjukkan pada Subbab 4.3. Langkah selanjutnya adalah menganalisis nilai kontribusi masing-masing variabel terhadap prediksi model. Analisis ini memanfaatkan nilai kontribusi yang dihasilkan oleh XGBoost, yang mencerminkan seberapa besar kontribusi setiap variabel dalam membentuk prediksi yang ditunjukkan pada Tabel 4.3. Seluruh variabel tetap digunakan sejak awal karena XGBoost mampu mengidentifikasi variabel-variabel yang kurang relevan melalui *feature importance*, sehingga proses seleksi manual tidak diperlukan di tahap awal.

Tabel 4.3: Nilai kontribusi variabel berdasarkan XGBoost

No	Variabel Bebas	Nilai Kontribusi
1	Jumlah Rumah Sakit Menurut Provinsi	0,620365
2	Jumlah Klinik Utama Teregistrasi Menurut Kepemilikan dan Provinsi	0,069493
3	Jumlah Klinik Pratama Menurut Kepemilikan dan Provinsi	0,066857
4	Kepadatan Penduduk per KM	0,052151
5	Area Bali dan Nusa Tenggara	0,043175
6	Persentase Penduduk Miskin	0,033292
7	Jumlah Sarana Produksi Bidang Kefarmasian Menurut Provinsi	0,032026
8	Tahun	0,018925
9	Jumlah Kabupaten/Kota yang Melakukan Pelayanan Terpadu (PANDU) di Setiap Puskesmas	0,011651
10	Rasio Tempat Tidur Rumah Sakit Menurut Provinsi	0,010206
11	Indeks Pembangunan Manusia (IPM)	0,008094
12	Rasio Puskesmas per Kecamatan	0,006203
13	Persentase Tempat dan Fasilitas Umum yang Dilakukan Pengawasan Sesuai Standar	0,005079
14	Area Sulawesi dan Maluku	0,004928
15	Persentase Kabupaten/Kota Melaksanakan Kebijakan Gerakan Masyarakat Hidup Sehat (GERMAS)	0,004088
16	Area Kalimantan	0,003882
17	Jumlah Alat Kesehatan Menurut Provinsi	0,003730
18	Persentase Tempat Pengolahan Pangan yang Memenuhi Syarat Sesuai Standar	0,001753
19	Persentase Kabupaten/Kota Melaksanakan Pembinaan Posyandu Aktif	0,001731
20	Persentase Puskesmas dengan Ketersediaan Vaksin Imunisasi Dasar Lengkap	0,001201
21	Area Sumatra	0,001173
22	Area Jawa	0,000000
23	Area Papua	0,000000

Berdasarkan Tabel 4.3, dapat dilihat bahwa tiga variabel dengan kontribusi terbesar merupakan

variabel yang berkaitan erat dengan fasilitas kesehatan, yaitu **Jumlah Rumah Sakit Menurut Provinsi**, **Jumlah Klinik Utama Teregistrasi Menurut Kepemilikan dan Provinsi**, dan **Jumlah Klinik Pratama Teregistrasi Menurut Kepemilikan dan Provinsi**. Dengan kata lain, semakin banyak fasilitas kesehatan yang tersedia, semakin tinggi pula jumlah pelaporan kasus DBD. Oleh karena itu, ketiga variabel tersebut menunjukkan nilai kontribusi terbesar dalam membentuk prediksi jumlah kasus DBD.

Berdasarkan hasil analisis, kontribusi **Jumlah Rumah Sakit Menurut Provinsi** mencapai sekitar 10 kali lipat lebih besar dibandingkan kontribusi **Jumlah Klinik Utama Teregistrasi Menurut Kepemilikan dan Provinsi** serta **Jumlah Klinik Pratama Teregistrasi Menurut Kepemilikan dan Provinsi**. Hal ini menunjukkan bahwa keberadaan rumah sakit di suatu wilayah memiliki kontribusi yang sangat signifikan terhadap pelaporan jumlah kasus DBD yang tercatat. Sebagai contoh, di Provinsi Sumatra Utara terdapat 211 rumah sakit dengan jumlah kasus DBD yang dilaporkan mencapai 5.623 kasus. Sementara itu, di Provinsi Jawa Barat terdapat 350 rumah sakit dan jumlah kasus DBD yang tercatat mencapai 8.732 kasus.

Selain itu, **Jumlah Klinik Pratama Teregistrasi Menurut Kepemilikan dan Provinsi** menunjukkan variasi yang cukup signifikan antarwilayah pada data. Beberapa daerah menunjukkan bahwa semakin banyak klinik pratama, semakin banyak pula kasus DBD yang dilaporkan. Namun, ada juga wilayah yang memiliki jumlah klinik pratama lebih sedikit namun mencatat laporan kasus DBD yang relatif tinggi. Sebagai contoh, di Provinsi Bengkulu terdapat 59 klinik pratama dengan jumlah kasus DBD sebesar 1.419 kasus, sedangkan di Provinsi Riau terdapat 160 klinik pratama, namun jumlah kasus DBD yang tercatat hanya 918 kasus. Hal ini menunjukkan bahwa jumlah klinik pratama tidak selalu berbanding lurus dengan jumlah kasus DBD, karena masih ada variabel bebas lain yang memiliki kontribusi yang lebih besar. Sama halnya dengan **Jumlah Klinik Utama Teregistrasi Menurut Kepemilikan dan Provinsi**, yang nilai kontribusi variabel tidak terlalu jauh berbeda dengan **Jumlah Klinik Pratama Teregistrasi Menurut Kepemilikan dan Provinsi**. Contohnya, di Provinsi Sumatra Barat terdapat 19 klinik utama dengan jumlah kasus DBD sebesar 2.263 kasus, sementara di Provinsi Riau terdapat 16 klinik utama tetapi jumlah kasus DBD tercatat lebih tinggi, yaitu 4.126 kasus.

Selain variabel fasilitas kesehatan, XGBoost juga menunjukkan kontribusi penting dari variabel sosiodemografis, seperti **Kepadatan Penduduk per KM**, **Persentase Penduduk Miskin**, **Indeks Pembangunan Manusia (IPM)**, serta **Area**, yang bersama-sama mencerminkan kondisi sosial ekonomi setiap wilayah. **Area Jawa** memiliki sekitar 56,1% dari total populasi Indonesia, dengan IPM tinggi yaitu 74,02, kepadatan penduduk 3.574 jiwa per km², dan persentase penduduk miskin hanya 8,32%. Hal ini menunjukkan kondisi sosial ekonomi yang lebih maju serta akses kesehatan yang lebih lengkap, yang turut berkontribusi terhadap tingginya jumlah kasus DBD yang tercatat. Sebaliknya, **Area Papua** memiliki kepadatan penduduk sangat rendah sekitar 9,91 jiwa per km², IPM sebesar 61,90, dan persentase penduduk miskin sebesar 25,05%. Kondisi ini mencerminkan keterbatasan dalam pembangunan dan akses terhadap layanan kesehatan, yang berkontribusi terhadap rendahnya jumlah kasus DBD yang tercatat. Namun, dalam Tabel 4.3, kontribusi **Area Jawa** dan **Area Papua** bernilai 0, karena karakteristik keduanya sudah sepenuhnya diwakili oleh variabel lain yang lebih dominan, sehingga tidak menambah informasi baru dalam model.

Berbeda dengan **Area Jawa** dan **Area Papua**, beberapa **Area** lain seperti **Area Bali dan Nusa Tenggara**, **Area Sulawesi dan Maluku**, **Area Kalimantan**, dan **Area Sumatra** tetap memberikan kontribusi meskipun tidak sebesar variabel utama lainnya. **Area Bali dan Nusa Tenggara** memiliki kepadatan penduduk 742,58 dan 269,95 jiwa per km², lebih tinggi dari wilayah lain selain **Area Jawa**. Sesuai Tabel 4.3, **Kepadatan Penduduk per KM** memiliki kontribusi sedikit lebih besar dari **Area Bali dan Nusa Tenggara**, menunjukkan kaitan yang erat. **Persentase Penduduk Miskin** juga memberikan kontribusi yang hampir setara, dengan rata-rata sekitar 12,84% di **Area Bali dan Nusa Tenggara**, lebih tinggi dibandingkan sebagian besar wilayah lain di Indonesia, kecuali **Area Papua**. Tingginya angka kemiskinan di dua provinsi tersebut menunjukkan tantangan sosial ekonomi, sehingga **Area Bali dan Nusa Tenggara** mencerminkan gabungan antara kepadatan dan kemiskinan. Sementara itu, **Area Sulawesi dan Maluku** dan **Area Kalimantan** memiliki kontribusi masing-masing sebesar 0,004928 dan 0,003882, kemungkinan karena nilai IPM yang berdekatan, yakni 71,76 dan 73,75. Kedekatan IPM ini mencerminkan tingkat pembangunan manusia yang serupa, didukung oleh jumlah fasilitas kesehatan yang relatif seimbang, yaitu 881 dan 767.

4.2 Hasil Analisis Menggunakan LSTM

Sama seperti pada Subbab 4.1, nilai terbaik dari MSE ditentukan berdasarkan kombinasi parameter yang diatur, seperti lapisan LSTM (*lstm_units*), lapisan Dropout (*dropout*), dan lapisan Dense (*dense_units*). Selain parameter yang disebutkan, ukuran batch (*batch_size*) juga dipertimbangkan karena berpengaruh terhadap hasil pelatihan model. Pada proses pelatihan, jumlah *epoch* ditetapkan sebanyak 100 untuk memastikan evaluasi performa model konsisten. Berikut adalah kombinasi parameter yang digunakan dalam proses pelatihan LSTM beserta kriteria pemilihan yang dirangkum pada Tabel 4.4. Setelah melakukan proses pelatihan, hasil terbaik model berdasarkan nilai MSE dirangkum pada Tabel 4.5.

Tabel 4.4: Kombinasi parameter yang digunakan dalam pelatihan LSTM

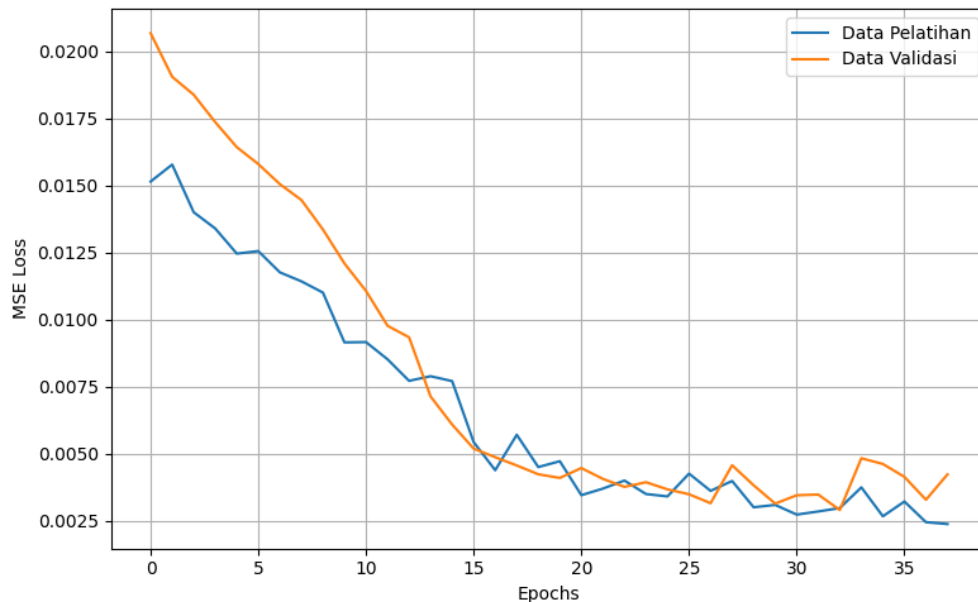
Parameter	Nilai yang Diuji
<i>lstm_units_1</i>	{32, 64, 128}
<i>lstm_units_2</i>	{16, 32, 64}
<i>dropout_1</i>	{0,2, 0,3, 0,4}
<i>dropout_2</i>	{0,1, 0,2, 0,3}
<i>dense_units</i>	{8, 16, 32}
<i>batch_size</i>	{8, 16, 32}
<i>epochs</i>	{100}

Tabel 4.5: Hasil terbaik LSTM berdasarkan nilai MSE

<i>lstm_1</i>	<i>lstm_2</i>	<i>dropout_1</i>	<i>dropout_2</i>	<i>dense</i>	<i>batch</i>	MSE
128	16	0,2	0,1	32	8	2.874.609,27
128	64	0,2	0,1	8	8	2.885.292,32
128	64	0,3	0,1	16	8	3.169.458,18

Setelah pelatihan menggunakan 729 kombinasi parameter dari Tabel 4.4, diperoleh tiga model dengan nilai MSE terbaik. Berdasarkan Tabel 4.5, model LSTM terbaik diperoleh dari kombinasi

parameter yang menghasilkan nilai MSE terendah, yaitu sebesar 2.874.609,27 pada data latih. Setelah menemukan nilai terbaik LSTM, maka langkah selanjutnya adalah menganalisis grafik data pelatihan dan data validasi berdasarkan hasil kombinasi parameter terbaik pada Tabel 4.5. Analisis grafik ini penting untuk memeriksa apakah model telah belajar dengan baik dan mampu merepresentasikan pola data secara akurat, sekaligus mengidentifikasi potensi *overfitting* atau *underfitting* yang ditunjukkan pada Gambar 4.1.



Gambar 4.1: Perbandingan MSE *loss* antara data pelatihan dan data validasi.

Berdasarkan Gambar 4.1, nilai *loss* pada data pelatihan dan data validasi sama-sama mengalami penurunan secara konsisten. Hal ini menunjukkan bahwa model mampu mempelajari pola dari data pelatihan dengan baik tanpa langsung mengalami *overfitting*. Setelah *epoch* ke-20, nilai *loss* terus menurun pada data pelatihan, sementara pada data validasi berada di kisaran 0,003 hingga 0,005. Rentang tersebut menunjukkan bahwa model cukup andal dalam melakukan prediksi terhadap data yang belum pernah dilihat sebelumnya. Mekanisme *early stopping* secara otomatis menghentikan pelatihan sebelum mencapai 100 *epoch*, tepat saat performa pada data validasi mulai memburuk. Pada *epoch* ke-35, nilai *loss* data validasi sedikit lebih tinggi dibandingkan data pelatihan, yang mengindikasikan gejala awal *overfitting*. Namun, karena pelatihan dihentikan tepat waktu, performa model tetap optimal dan tidak mengalami penurunan signifikan pada data validasi. Dengan demikian, proses pelatihan yang berlangsung secara terkontrol, disertai penerapan mekanisme *early stopping*, mampu menghasilkan model yang tetap akurat saat digunakan untuk memprediksi data yang belum pernah dilihat.

4.3 Perbandingan Evaluasi XGBoost dan LSTM

Setelah menemukan hasil terbaik pada XGBoost dan LSTM pada data latih, selanjutnya perlu dibandingkan performa kedua model pada data uji. Hal ini penting untuk mengetahui seberapa baik model dapat bekerja dan memberikan prediksi yang tepat pada data yang belum pernah dilihat sebelumnya. Berdasarkan Tabel 4.6, dapat disimpulkan bahwa XGBoost belum menunjukkan

performa yang baik pada data latih, dengan nilai MSE sebesar 9.016.559,00, yang menunjukkan bahwa model belum mampu menangkap data latih dengan baik. Namun, performa model ini justru meningkat secara signifikan saat diuji pada data uji, dengan nilai MSE menurun menjadi 4.833.464,00, yang menunjukkan bahwa model mampu memprediksi data yang belum pernah dilihat dengan cukup baik, meskipun belum optimal dalam mempelajari data latih.

Sementara itu, LSTM memperoleh nilai MSE pada data latih sebesar 2.874.609,27, yang menunjukkan kemampuannya mempelajari informasi penting tanpa *overfitting*. Pada data uji, nilai MSE mengalami sedikit kenaikan menjadi 3.860.943,84, namun tetap menunjukkan performa yang baik dan lebih rendah dibandingkan XGBoost. Meskipun terdapat kenaikan nilai MSE pada data uji, selisih tersebut masih tergolong kecil sehingga LSTM tidak mengalami *overfitting* dan tetap mampu memprediksi data yang belum pernah dilihat dengan baik. Hal ini menunjukkan bahwa LSTM memiliki kemampuan yang lebih baik dalam mempelajari data yang belum pernah dilihat, sehingga prediksi yang dihasilkan tetap konsisten.

Secara matematis, LSTM memiliki *cell state* (c_t) dan *hidden state* (h_t) yang memungkinkan model untuk mengingat informasi sebelumnya. Hal ini membuat LSTM sangat cocok untuk data DBD yang disusun secara berurutan, sehingga mampu menangkap pola jangka panjang dan memberikan prediksi yang lebih akurat. Namun, kelemahannya adalah membutuhkan data yang cukup banyak, waktu pelatihan yang lebih lama, dan hasilnya agak sulit untuk dijelaskan.

Sebaliknya, XGBoost tidak dirancang untuk mengenali hubungan dari waktu ke waktu karena hanya menggunakan pohon keputusan berdasarkan data yang bersifat tetap. Meskipun begitu, XGBoost memiliki keunggulan dalam pelatihan yang cepat, hasilnya mudah dijelaskan melalui kontribusi setiap variabel, dan fleksibel dalam menangani berbagai jenis data. Dalam kasus data DBD, XGBoost lebih tepat digunakan untuk melihat pengaruh antarvariabel dalam satu waktu, tetapi kurang efektif dalam memahami pola dari tahun ke tahun.

Dengan demikian, performa kedua model sangat dipengaruhi oleh struktur data. LSTM lebih unggul dalam mengenali pola berulang dari tahun ke tahun, sedangkan XGBoost lebih kuat dalam menganalisis hubungan antarvariabel pada satu periode waktu. Meski LSTM berisiko mengalami kenaikan MSE saat data uji memiliki karakteristik berbeda dari data latih, selama selisihnya kecil, model tetap dianggap mampu menggeneralisasi tanpa *overfitting*. Berdasarkan evaluasi, LSTM menunjukkan kinerja yang lebih stabil dan konsisten, sehingga lebih akurat dan layak digunakan untuk meramalkan jumlah kasus DBD, termasuk prediksi pada tahun-tahun berikutnya di masing-masing provinsi.

Tabel 4.6: Perbandingan evaluasi nilai MSE pada XGBoost dan LSTM

No	Model	Data Latih	Data Uji
1	LSTM	2.874.609,27	3.860.943,84
2	LSTM	2.885.292,32	3.453.606,94
3	LSTM	3.169.458,18	4.089.321,99
4	XGBoost (10-fold)	9.016.559,00	4.833.464,00
5	XGBoost (5-fold)	10.812.922,00	5.398.128,00
6	XGBoost (3-fold)	11.848.894,00	5.507.332,00

BAB 5

KESIMPULAN DAN SARAN

Pada bab ini akan disampaikan rangkuman hasil penelitian berupa kesimpulan yang diperoleh serta beberapa saran yang dapat menjadi masukan untuk pengembangan penelitian selanjutnya.

5.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa kesimpulan yang dapat diambil sebagai berikut:

1. Model pembelajaran mesin memanfaatkan data historis dan variabel-variabel sosiodemografis dengan cara mempelajari pola hubungan antara jumlah kasus DBD sebelumnya dan variabel-variabel sosiodemografis. Untuk mendukung proses pembelajaran ini, digunakan kombinasi parameter tertentu yang diatur secara sistematis. Pengaturan parameter ini membantu model dalam menangkap pola data dengan tepat. Selain itu, kombinasi parameter membantu dalam meminimalkan kesalahan dan menjaga model tetap akurat terhadap data baru.
2. Berdasarkan variabel-variabel yang digunakan dari hasil analisis XGBoost, variabel-variabel yang paling berkontribusi terhadap prediksi jumlah kasus DBD adalah **Jumlah Rumah Sakit Menurut Provinsi, Jumlah Klinik Utama Teregistrasi Menurut Kepemilikan dan Provinsi, dan Jumlah Klinik Pratama Menurut Kepemilikan dan Provinsi**. Hal ini menunjukkan bahwa ketersediaan fasilitas kesehatan memiliki kontribusi penting dalam menangani kasus DBD. Selain itu, variabel sosiodemografis seperti **Kepadatan Penduduk per KM, Persentase Penduduk Miskin, dan Indeks Pembangunan Manusia (IPM)** juga berkontribusi dalam pembentukan model, meskipun tidak sebesar variabel fasilitas kesehatan. Variabel yang berhubungan dengan wilayah, misalnya **Area**, tetap memiliki kontribusi, meskipun hanya sebagai pelengkap karena sudah diwakili oleh variabel lain yang lebih informatif.
3. Berdasarkan nilai MSE, XGBoost belum menunjukkan performa yang baik pada data latih, dengan nilai MSE sebesar 9.016.559,00, yang mengindikasikan bahwa model belum mampu menangkap data latih secara optimal. Namun, saat diuji pada data baru, performa XGBoost justru meningkat dengan nilai MSE yang lebih rendah, yaitu sebesar 4.833.464,00, sehingga dapat memprediksi data baru meskipun performanya tidak stabil. Sementara itu, LSTM menunjukkan performa yang lebih baik dan konsisten, dengan nilai MSE pada data latih sebesar 2.874.609,27. Pada data uji, LSTM memperoleh nilai MSE sebesar 3.860.943,84, yang

nilainya masih mendekati dengan data latih, sehingga menunjukkan kemampuan model dalam memprediksi data yang belum pernah dilihat sebelumnya lebih akurat. Dengan demikian, LSTM menunjukkan performa yang lebih baik secara keseluruhan dan mampu menghasilkan prediksi yang lebih akurat dibandingkan XGBoost.

5.2 Saran

Berdasarkan penelitian yang telah dilakukan, berikut adalah beberapa saran yang dapat dijadikan pertimbangan untuk penelitian selanjutnya.

1. Penambahan variabel lain yang relevan, seperti data iklim (suhu rata-rata, curah hujan, dan kelembaban) serta kondisi lingkungan (misalnya sanitasi dan saluran air), agar model dapat mempelajari pola penyebaran DBD dengan lebih lengkap dan meningkatkan akurasi prediksi.
2. Data yang digunakan sebaiknya memiliki rentang waktu yang lebih panjang serta frekuensi data yang lebih detail, misalnya data mingguan atau harian agar model dapat menangkap pola musiman dengan lebih akurat.

DAFTAR REFERENSI

- [1] Wardhana, R. G., Wang, G., dan Sibuea, F. (2023) Penerapan machine learning dalam prediksi tingkat kasus penyakit di Indonesia. *Journal of Information System Management (JOISM)*, **5**, 40–45.
- [2] Nurillah, R. A. S., Imrona, M., dan Alamsyah, A. (2021) Prediksi pola penyebaran penyakit DBD di Kota Pagar Alam menggunakan LSTM. *eProceedings of Engineering*, **8**.
- [3] Goodfellow, I., Bengio, Y., dan Courville, A. (2016) *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>.
- [4] James, G., Witten, D., Hastie, T., dan Tibshirani, R. (2013) *An Introduction to Statistical Learning: with Applications in R*. Springer.
- [5] Breiman, L. (2001) Random forests. *Machine Learning*, **45**, 5–32.
- [6] Liparas, D., HaCohen-Kerner, Y., Moutzidou, A., Vrochidis, S., dan Kompatsiaris, I. (2014) News articles classification using random forests and weighted multimodal features. *Proceedings of the 7th Information Retrieval Facility Conference (IRFC 2014)*. Springer.
- [7] Natekin, A. dan Knoll, A. (2013) Gradient boosting machines, a tutorial. *Frontiers in Neurorobotics*, **7**, 21.
- [8] Ali, Z., Abduljabbar, Z., Tahir, H., Sallow, A., dan Almufti, S. (2023) Exploring the power of extreme gradient boosting algorithm in machine learning: a review. *Academic Journal of Nawroz University*, **12**, 320–334.
- [9] Guo, R., Liu, Y., Zhao, Z., Zhao, J., Wang, J., dan Cai, W. (2022) Research on degradation state recognition of axial piston pump under variable rotating speed. *Processes*, **10**, 1078.
- [10] Hu, T. dan Song, T. (2019) Research on xgboost academic forecasting and analysis modeling. *Journal of Physics: Conference Series* 012091. IOP Publishing.
- [11] Aggarwal, C. C. (2018) *Neural Networks and Deep Learning: A Textbook*. Springer.
- [12] Kementerian Kesehatan Republik Indonesia (2018) *Profil Kesehatan Indonesia 2018*. Kemenkes RI.
- [13] Kementerian Kesehatan Republik Indonesia (2019) *Profil Kesehatan Indonesia 2019*. Kemenkes RI.
- [14] Kementerian Kesehatan Republik Indonesia (2020) *Profil Kesehatan Indonesia 2020*. Kemenkes RI.
- [15] Kementerian Kesehatan Republik Indonesia (2021) *Profil Kesehatan Indonesia 2021*. Kemenkes RI.
- [16] Kementerian Kesehatan Republik Indonesia (2022) *Profil Kesehatan Indonesia 2022*. Kemenkes RI.